AI Tutor: An Evidence-Based Adaptive Learning System with Deliberate Scaffolding Withdrawal

> **Draft report for Review 3 (Implementation, Testing and Draft Report), 23 October 2026.** Status as of 19 October 2026. Prepared to the MCA-269 "Format of Project Report" and the Annexure-IV chapter scheme. Typesetting rules (Times New Roman 12, 1.5 spacing, margins 3.0/2.0/2.54/2.54 cm, blank page before each chapter) are applied in the final printed copy.

Cover Page (Annexure-I)

AI Tutor: An Evidence-Based Adaptive Learning System with Deliberate Scaffolding Withdrawal

Submitted in partial fulfillment of the requirements for the award of the degree of **Master of Computer Applications** to **Guru Gobind Singh Indraprastha University, Delhi**

Guide	Submitted by
Dr. Chetna Laroia	Ashwani Kumar Puri (04914004425)
	Aman Dogra (01614004425)
	Aditi Tanwar (01814004425)

JAGAN INSTITUTE OF MANAGEMENT STUDIES 3, Institutional Area, Sector-5, Rohini Batch (2025–2027)

Certificate (Annexure-II)

We, 1. Ashwani Kumar Puri (04914004425), 2. Aman Dogra (01614004425) and 3. Aditi Tanwar (01814004425), certify that the Minor-III Project Report (MCA-269) entitled "AI Tutor: An Evidence-Based Adaptive Learning System with Deliberate Scaffolding Withdrawal" is done by us and it is an authentic work carried out by us at Jagan Institute of Management Studies, Rohini. The matter embodied in this project work has not been submitted earlier for the award of any degree or diploma to the best of our knowledge and belief.

Signature of the Students: _____ Date: _____

Certified that the Minor-III Project Report (MCA-269) entitled "AI Tutor: An Evidence-Based Adaptive Learning System with Deliberate Scaffolding Withdrawal" done by the above students is completed under my guidance.

Signature of the Guide: _____ Date: _____ Name of the Guide: Dr. Chetna Laroia Designation: _____

Countersigned: Principal/Director _____

Acknowledgement

We are grateful to our guide, **Dr. Chetna Laroia**, for her guidance and feedback at every review of this project. We thank the faculty of the MCA department of Jagan Institute of Management Studies for the knowledge of databases, software engineering and web development this project is built on, and the institute for the facilities and the opportunity. We also thank the authors of the research papers listed in the references, whose findings on learning shaped the design of this system.

Ashwani Kumar Puri · Aman Dogra · Aditi Tanwar

Table of Contents

	Topic	
	Certificate · Acknowledgement · List of Tables/Figures/Symbols · Synopsis	
1	**Problem Formulation**: 1.1 Introduction about the organisation · 1.2 Introduction about the problem · 1.3 Present state of the art · 1.4 Need of computerization · 1.5 Proposed software · 1.6 Importance of the work	
2	**System Analysis**: 2.1 Feasibility study · 2.2 Analysis methodology and system planning (PERT) · 2.3 Choice of the platforms	
3	**System Design**: 3.1 Design methodology · 3.2 UML modelling · 3.3 Database design (ERD, DFD) · 3.4 Input design · 3.5 Output design · 3.6 Code design and development	
4	**Testing and Implementation**: 4.1 Testing methodology · 4.2 Test data and test cases · 4.3 Test reports and debugging · 4.4 Implementation manual · 4.5 Implementation · 4.6 Users training · 4.7 Post-implementation maintenance	
5	**Conclusion and References**: 5.1 Conclusion · 5.2 System specifications · 5.3 Limitations · 5.4 Future scope · 5.5 References	
6	**Annexures**: A-1 Menu flow · A-2 Structure chart · A-6 Decision tables · A-7 Data dictionary · A-8 Test reports · A-9 Sample inputs · A-10 Sample outputs · A-11 Coding	

Page numbers are added in the printed copy.

List of Tables

No	Title
1	Problems with the existing (manual) way of learning
2	Functional modules and their users
3	Feasibility summary with measured costs
4	Software and hardware used
5	Verdicts and how each one is decided
6	Mastery formula factors
7	Fade stages
8	Automated test suites and results
9	Defects found and fixed during testing
10	Data dictionary (key tables)
11	Placement test rules

No	Title
12	Hint ladder and where each hint comes from
13	Exercise generation: how a job runs

List of Figures

No	Title
1	System architecture
2	Use case diagram
3	Class diagram: domain model
4	Class diagram: services and adapters
5	Sequence diagram: register, then log in
6	Sequence diagram: submit code
7	State diagram: submission
8	Activity diagram: next-step selection
9	ER diagram: overview, and attempts and execution
10	DFD level 0, level 1, and level 2 (execute and grade)
11	System planning (PERT/Gantt)
12	Deployment diagram
13	Menu flow diagram
14	Sequence diagram: request a hint
15	Activity diagram: placement test

List of Symbols and Abbreviations

Symbol	Meaning
AC, WA, CE, RE, TLE, MLE, SE	Verdicts: accepted, wrong answer, compile error, runtime error, time limit exceeded, memory limit exceeded, sandbox error
λ (lambda)	Decay rate of a skill per day
e	Euler's number; effective mastery = score × e^(−λ × days)
Rung	Exercise difficulty level 1–5 (Predict → Build)
API, JWT, ORM	Application programming interface, JSON Web Token, object-relational mapper

Symbol	Meaning
DPDP	Digital Personal Data Protection Act, 2023 (India)
E2B	Cloud sandbox service that runs learner code in isolated micro-VMs
LLM	Large language model (Google Gemini in this project)

Synopsis

1. Title of Project. AI Tutor: An Evidence-Based Adaptive Learning System with Deliberate Scaffolding Withdrawal. In one line: a tutor whose objective is to make itself unnecessary.

2. Problems with the Existing System. Today a self-taught programmer learns "manually": they pick a video course, watch it in order, copy the instructor's code, tick lessons as complete and, when stuck, ask an AI assistant for the answer. Nothing records what they can actually *write*. Completion is measured instead of capability, the order is the same for everyone, help is unlimited and never withdrawn, and nothing schedules revision before a skill is forgotten.

3. Description of the Proposed System. A web-based, online system that keeps a separate **reading** and **writing** mastery score for every skill, updates them **only from code that was actually executed and graded**, chooses the next step from a prerequisite graph, and **withdraws help on a schedule** until the learner can work alone. Learner code runs in an isolated cloud sandbox; an LLM writes lessons, exercises and hints, but never grades and never decides what comes next.

4. Functional Modules and Users. Users: **Learner**, **Author** (maintains the skill graph and templates), **Admin** (analytics). Thirteen modules: M1 Accounts and consent, M2 Skill graph, M3 Diagnostic, M4 Mastery model, M5 Recommender, M6 Exercises, M7 Execution and grading, M8 Hint ladder, M9 Sessions, M10 Revision, M11 Progress, M12 Scaffolding fade, M13 Analytics.

5. Tools/Platforms.

- **5.1 Hardware:** development on a laptop with a 4-core or better processor, 8–16 GB RAM, 256 GB SSD. Production: one cloud VM (4 ARM cores, 24 GB RAM, Oracle Cloud Always Free) plus managed sandboxes.
- **5.2 Software:** Windows 11 / Ubuntu 24.04; front end Next.js 16, React 19, TypeScript, Monaco editor; back end Node.js 22, Express 5, TypeScript; database PostgreSQL 16 with Drizzle ORM; Redis 7 with BullMQ; E2B sandboxes with clang 19 (C++20); Google Gemini API; Docker, pnpm, Git/GitHub, Vitest.

6. Methodology. 6.1 Incremental and iterative development. 6.2 It suits a system whose correctness is partly statistical: each increment produces a working system whose predictions can be checked against real learner performance, and the design decisions are recorded as they are made (architecture decision records), so later increments build on settled ground.

7. Future Scope. More courses (Java, Python, frontend, backend), a mobile app, colleges as institutional customers, live calibration of the mastery model from real learners, and spoken

explanation assessment.

CHAPTER 1: PROBLEM FORMULATION

1.1 Introduction about the Organisation

The project is carried out at **Jagan Institute of Management Studies (JIMS), Rohini**, as the MCA-269 Minor Project of the 2025–2027 batch, under the guidance of Dr. Chetna Laroia. It is not built for an outside company. The team (Ashwani Kumar Puri: backend, architecture and database; Aman Dogra: frontend; Aditi Tanwar: frontend and UI/UX) intends to continue it as a product under the name **Nidanos** after the course, which is why commercial concerns (licensing, data protection law, running cost) were treated as real requirements from the start.

1.2 Introduction about the Problem

Learning to program has become easy to *start* and hard to *finish*. A learner can watch hours of video and follow along successfully, yet face a blank editor and be unable to write a working function. The problem was first observed by a team member while learning a web stack under time pressure, and later measured on a pilot learner (Learner A, anonymised): **10 of 11** on reading code, **0 of 4** on writing it. A single "progress" number would have averaged these into about 65% and hidden the real situation completely.

Three causes stand out:

1. **Completion is measured instead of capability.** Platforms record what was watched or ticked, not what the learner can produce.
2. **Help is unlimited and never withdrawn.** An AI assistant asked for code gives code; productivity rises, capability does not.
3. **Nobody keeps a model of the learner.** Without one, the next step is chosen by playlist order, not by what this learner needs.

1.3 Present State of the Art

Kind of system	Examples	What it does well	What it lacks for our problem
Video course platforms	Coursera, Udemy, free video courses	Wide content, low cost	No learner model; completion measured, not capability

Kind of system	Examples	What it does well	What it lacks for our problem
Interactive coding platforms	Codecademy, Scrimba	Learner types code	Fixed sequence, heavy scaffolding that never fades
Practice and assessment	LeetCode, HackerRank, Exercism	Real graded code	No teaching, no learner model, no adaptive path
Spaced repetition	Anki, Quizlet	Schedules revision	Recall of facts, not writing code
General AI assistants	ChatGPT, GitHub Copilot	Explain anything	Give answers freely; no memory of what the learner can do
AI tutoring products	Khanmigo, Duolingo Max	Guided hints	Not focused on writing code; no separate reading/writing model

The research behind our design:

- **Bloom (1984)**: one-to-one tutoring with mastery learning lifts the average student about two standard deviations, so we aim at mastery-based progression.
- **Roediger & Karpicke (2006)**: recall builds memory and re-reading builds false confidence, so mastery moves only on produced evidence.
- **Bjork & Bjork**: desirable difficulties, so help is limited on purpose.
- **Cepeda et al. (2006)**: the spacing effect, so mastery decays and revision is scheduled.
- **Sweller**: worked examples help beginners, but the effect reverses with expertise, so support fades rung by rung.
- **Corbett & Anderson (1994)**: knowledge tracing. We keep its idea of estimating hidden skill from noisy evidence, but use two scores and add forgetting.
- **VanLehn (2011)**: step-level tutoring works best, so hints respond to the current attempt.
- **Bastani et al. (2025)**: unrestricted AI help made students score 17% lower once the AI was removed; restricted, hint-only help did not.

1.4 Need of Computerization

The "manual" way (a learner, a playlist and an unrestricted AI chat) cannot do what the problem needs:

- **Grading real code** needs a compiler, hidden tests and time and memory limits, applied identically every time. Done by hand, it is slow and inconsistent.

- **Keeping two mastery scores per skill**, decaying them daily and adjusting them after every submission, is arithmetic over hundreds of events per learner.
- **Choosing the next step** from a prerequisite graph and current mastery is a query, not a guess.
- **Enforcing limits on help** (hint ceilings that fade) only works if the server, not the learner, holds the rules.

1.5 Proposed Software / Project

A web application in which the learner:

1. takes a short adaptive placement test, so that skills they already know are fast-tracked;
2. is given the next step (skill, level and exercise) by fixed rules, in study sessions they can pause and resume;
3. opens an exercise matched to their level (five rungs, from *predict the output* to *build from a blank page*);
4. writes code in an in-browser editor and submits it;
5. has it compiled and run in an isolated sandbox against hidden tests, getting one of seven verdicts;
6. sees their reading and writing mastery change from that evidence;
7. asks for hints one level at a time, and gets less help over time as they succeed with few hints.

The system is **deterministic where it matters**: grading, mastery, placement and next-step choice are fixed rules in code. The LLM only produces content: exercises and stored hints are verified and frozen before any learner sees them, and live hints about the learner's own code pass a filter that blocks anything resembling the solution.

1.6 Importance of the Work

- **For learners**, especially students preparing Data Structures and Algorithms in C++ for placements: an honest picture of what they can write, and practice that builds independence instead of dependence.
- **For institutions**: a measure of real coding ability rather than course completion.
- **Technically**, the project combines several areas from the semester: database design (42-table schema with integrity constraints), software engineering (layered modules, ADRs,

automated tests), security (authentication, sandboxing, India's DPDP Act 2023) and AI (LLM content under strict guardrails).

CHAPTER 2: SYSTEM ANALYSIS

2.1 Feasibility Study

2.1.1 Technical Feasibility

The hard technical parts were proven early by building them, not just by planning them.

Risk	How it was settled	Evidence
Running untrusted C++ safely	E2B cloud sandboxes (Firecracker micro-VMs), **one fresh offline sandbox per submission**, killed after use	All six verdict types reproduced on the real service (section 4.3)
Judging memory and time limits	Our own judge: `timeout` for time, `prlimit` for address space, GNU `time` for peak memory	MLE, TLE and RE detected correctly
Speed	Measured each step and cut the network round trips from 2 per test to 3 per submission	5–15 s down to about 6 s per submission
Data integrity	PostgreSQL constraints: CHECK ranges, partial unique indexes, foreign keys	16 database-rule tests
Placing learners fairly	A placement walk over the skill graph with a check question after a possible slip	All 711 possible knowledge states placed exactly in simulation (section 4.2)
Keeping the AI from giving answers	A filter compares every hint with the reference solution; caps per learner and per month	Unit tests with real leak attempts (section 4.2)
Trusting AI-written exercises	Stored only after the sandbox proves them: the reference passes, the starter fails, and 3 wrong answers fail	Tests with a fake model and sandbox, including weak tests being rejected (section 4.2)
Team skills	The stack (TypeScript, React, Node, SQL) matches the semester courses	Working code in the repository

2.1.2 Economic Feasibility

The budget is a student budget, so every running cost was measured or quoted, not assumed.

Item	Choice	Cost
Hosting (API, worker, database, Redis)	Oracle Cloud Always Free VM in an India region; fallback Azure for Students credit	₹0 per month
Web hosting	Vercel free plan during the college phase	₹0
Code execution	E2B, billed per second while a sandbox runs; $100 starting credit	about ₹0.01 per submission
Exercise generation	Gemini 3.8 Flash (3.1 Pro on a third try), once per exercise, then frozen; its own cap of ₹300 per month	about ₹1.5 per exercise; ₹110–150 for skills 6–20
Live tutor hints	Gemini 3.5 Flash Lite, capped at 20 per learner per day and ₹500 per month in total	about ₹0.08 per hint (₹0.01 measured on short test prompts)

One cost was found only when paying: a new Google Cloud billing account in India must prepay **₹3,000** in total before the paid Gemini tier activates (₹500 paid on 18 October, ₹2,500 due on 7 November). Until then the key stays on the free tier, where prompts may be used by Google, so live hints are switched off by a setting until billing is active (7 November 2026, section 4.5).

One design decision is a direct result of this study. The first design kept one sandbox alive for a whole learner session. Because E2B bills every second a sandbox is alive, that would cost about **₹4.60** for a 30-minute session. Creating a fresh sandbox per submission costs about **₹0.25** for the same session, roughly 18 times less, so the design was changed (ADR-0003).

2.1.3 Operational Feasibility

- **Learners** need only a browser. There's no installation; the code editor and compiler run online.
- **The team** can run the whole system locally with three commands (section 4.4). The server is the same Docker Compose setup, so operating it needs no special skills.
- **Authors** add skills and exercises as data (seed files), not code changes.

2.1.4 Other Feasibility Dimensions

- **Legal.** The system stores personal data, so it is designed for India's **Digital Personal Data Protection Act, 2023**:
    - consent at sign-up, recorded with the policy version;
    - separate opt-ins for AI training and research, adults only;
    - learners under 18 are held until a parent approves;

    - learner code is never sent to the AI with a name or email;
    - data is stored in India.
- **Licensing.** No third-party course content is copied. Skill names and descriptions are our own, and external tutorials appear only as links.
- **Schedule.** All 23 MVP endpoints were live by Review 3 (section 4.5); the frontend, deployment, the first live AI runs (once billing is active on 7 November) and the pilot are planned up to 10 November 2026.

2.2 Analysis Methodology

Model: incremental and iterative. Each increment adds one working, tested capability on top of the previous one:

1. Schema and authentication.
2. Consent and data protection.
3. Sandbox and judge.
4. Submissions and queue.
5. Mastery engine.
6. Exercises and attempts.
7. Help fading.
8. Template exercise bank, verified in the sandbox.
9. Next-step recommender.
10. Placement test.
11. Study sessions and the progress view.
12. Admin analytics.
13. LLM adapter and hints.
14. Exercise generation.

Every increment ended with automated tests and a real end-to-end run before the next one began. Larger designs (the placement test, hints, generation) were also reviewed by adversarial checks that tried to break them, and the holes they found were fixed before the code was committed (Table 9).

Analysis tools used:

- **Requirements:** the SRS from Phase 1, numbered as FR-Mx-yy and NFR-xx.
- **Decisions:** each one recorded as an Architecture Decision Record (ADR-0001 to ADR-0016), with the options compared and the reason for the choice. For example, ADR-0003 compares Judge0, Piston and E2B on compiler version, memory-limit support, network isolation and cost.
- **Measurement before optimisation:** timings were measured step by step before anything was changed.

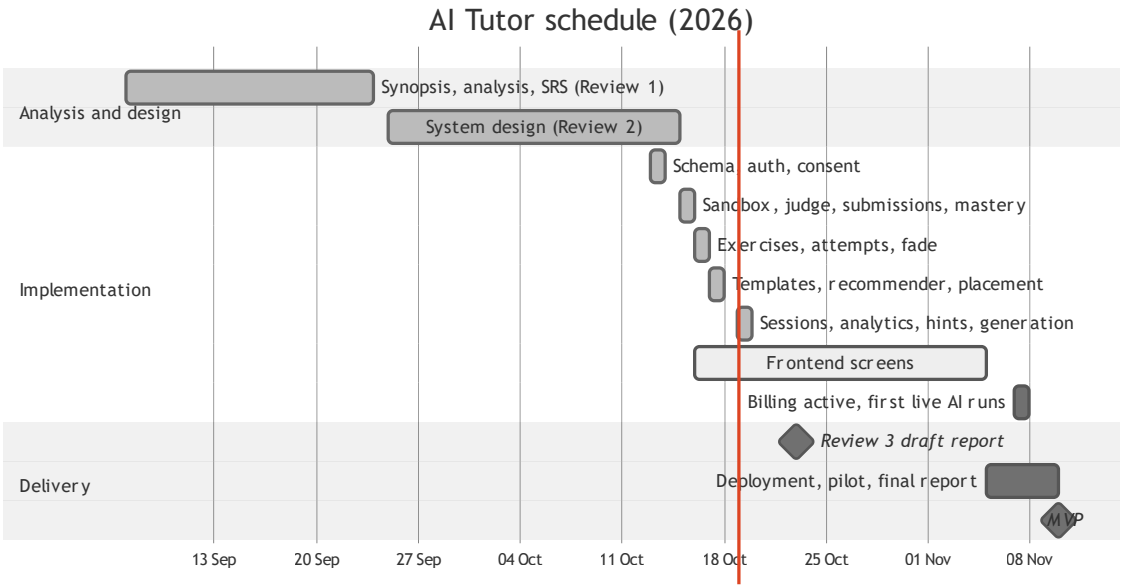
Pilot baseline. The planned "Phase 0" (running the whole loop on paper with a learner before coding) was **not carried out**. It was replaced by a measured baseline of one anonymised pilot learner (Learner A: reading 10/11, writing 0/4), which confirmed the central design choice of separate reading and writing scores. A pilot with real learners on the finished system is planned before the final report.

System Planning (PERT)

Activity	Depends on	Dates	Status
A. Synopsis, problem analysis, literature survey, SRS	–	7–24 Sep	Done (Review 1)
B. System design: DFD, ER, UML, ADRs	A	25 Sep–15 Oct	Done (Review 2)
C. Database schema and migrations	B	13 Oct	Done
D. Authentication and DPDP consent	C	13–14 Oct	Done
E. Sandbox and judge	C	15 Oct	Done
F. Submissions, queue, mastery	D, E	15 Oct	Done
G. Exercises, attempts, help fading	F	16 Oct	Done
H. Frontend screens	D	16 Oct–5 Nov	In progress
J. Template bank, recommender, placement test	G	17–18 Oct	Done
L. Sessions, progress, analytics	J	19 Oct	Done
I1. LLM adapter and hints	G	19 Oct	Done (live calls wait for billing)
I2. Exercise generation	I1	19 Oct	Done (live runs wait for billing)

Activity	Depends on	Dates	Status
M. Gemini billing activation	–	7 Nov	Planned (₹2,500 of the ₹3,000 prepayment left)
I3. First live AI runs: frozen hints, generated exercises for skills 6–20	I2, M	7–8 Nov	Planned
K. Deployment, pilot, final report	H, I3, L	5–10 Nov	Planned

Critical path: A → B → C → E → F → G → J → I1 → I2 → M → I3 → K. The sandbox and grading chain was the riskiest part, so it was built first. The billing activation (M) is now the one outside dependency on the path.



2.3 Choice of the Platforms

2.3.1 Software Used

Layer	Choice	Why this, for this project
Language	TypeScript (end to end)	One language for frontend and backend; shared types catch contract mistakes at compile time
Front end	Next.js 16, React 19, Monaco editor	The same editor as VS Code; a built-in proxy to the API means the same origin, so no CORS issues and httpOnly cookies work

Layer	Choice	Why this, for this project
Back end	Node.js 22, Express 5	Most backend work is waiting on the database, sandbox or LLM, which Node's event loop handles cheaply. Express 5 forwards async errors to one handler
Validation	zod 4	Every request body is checked against one schema, and types are inferred from it
Database	PostgreSQL 16 with Drizzle ORM	Constraints, transactions and row locks keep mastery consistent; Drizzle gives typed queries and versioned migrations
Queue	Redis 7, BullMQ	Running code is slow, so it is queued: retries, rate limits, and the API stays fast
Code execution	E2B sandboxes, Debian 13, clang 19, C++20	Managed isolation with internet off; a current compiler; per-second billing
AI	Google Gemini (paid tier)	Cheapest capable models; paid-tier data is not used for training; hidden behind one adapter
Security	argon2id, jose (JWT), httpOnly cookies, express-rate-limit, helmet	Passwords hashed, short-lived tokens, CSRF origin check, rate limits
Testing	Vitest, Supertest	Fast unit tests and HTTP-level integration tests on a separate test database
Tooling	pnpm workspaces, Docker Compose, Git/GitHub	One repository for web, API and shared types; identical local setup for every team member

2.3.2 Hardware Used

Purpose	Specification
Development machine	Windows 11 laptop, multi-core processor, 8–16 GB RAM, SSD
Staging and backup server	Older laptop running Ubuntu Server 24.04 LTS (AMD Ryzen 5 4600H, 8 GB RAM)
Production server (planned)	Oracle Cloud Always Free ARM VM: up to 4 cores, 24 GB RAM, India region
Sandboxes	E2B micro-VMs: 2 vCPU, 512 MiB RAM each, created per submission
Learner's device	Any modern browser; no installation

CHAPTER 3: SYSTEM DESIGN

3.1 Design Methodology

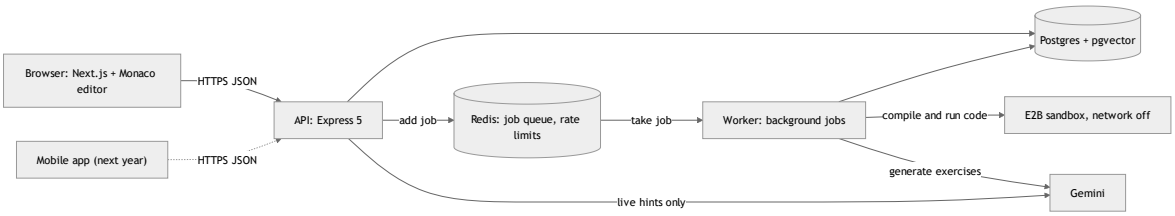
The system is a **modular monolith**: one backend, divided into 13 modules (M1–M13). Each module has the same four layers:

```
routes → controllers → services → repositories → database
(URL)    (HTTP in/out)  (rules)    (SQL only)
```

Five rules hold the design together:

1. **Modules talk through services, never through each other's tables.** For example, the execution module (M7) asks the exercises module (M6) for an exercise through `getRunnableExercise()`; it never reads M6's tables.
2. **One writer per state.** Only M4 writes mastery, and only M12 writes fade stages.
3. **Deterministic core, LLM at the edge.** Grading, mastery and next-step choice are fixed code. The LLM only produces content, which is verified and frozen first.
4. **Server-owned state.** Hint level, hint ceiling, fade stage, mastery and verdicts live only in the database, so the client can't tamper with them.
5. **Slow work is queued.** The API answers in milliseconds; running code happens in a background worker.

Figure 1: System architecture



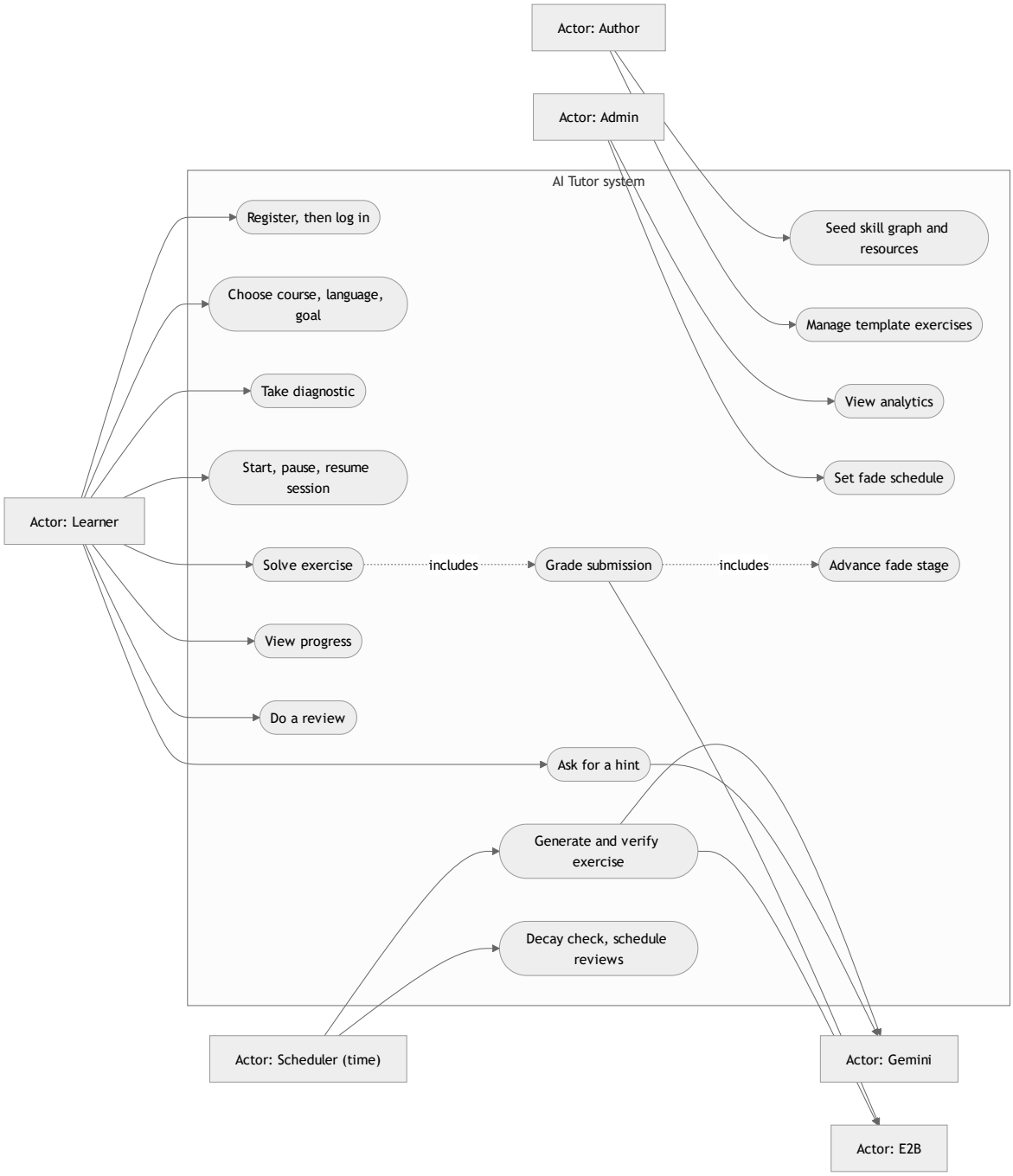
3.2 UML Modelling

3.2.1 Use Case

The learner registers, takes a diagnostic, solves exercises, asks for hints and views progress. The author maintains the skill graph and template exercises; the admin views analytics. Two non-human

actors matter: the **scheduler** (mastery decays with time, not only with clicks) and the **sandbox/LLM services**.

Figure 2: Use case diagram



3.2.2 Sequence Diagrams

Figure 5: Register, then log in. Registration does not log the user in. The user logs in separately and receives a 15-minute access token plus a rotating 30-day refresh token.

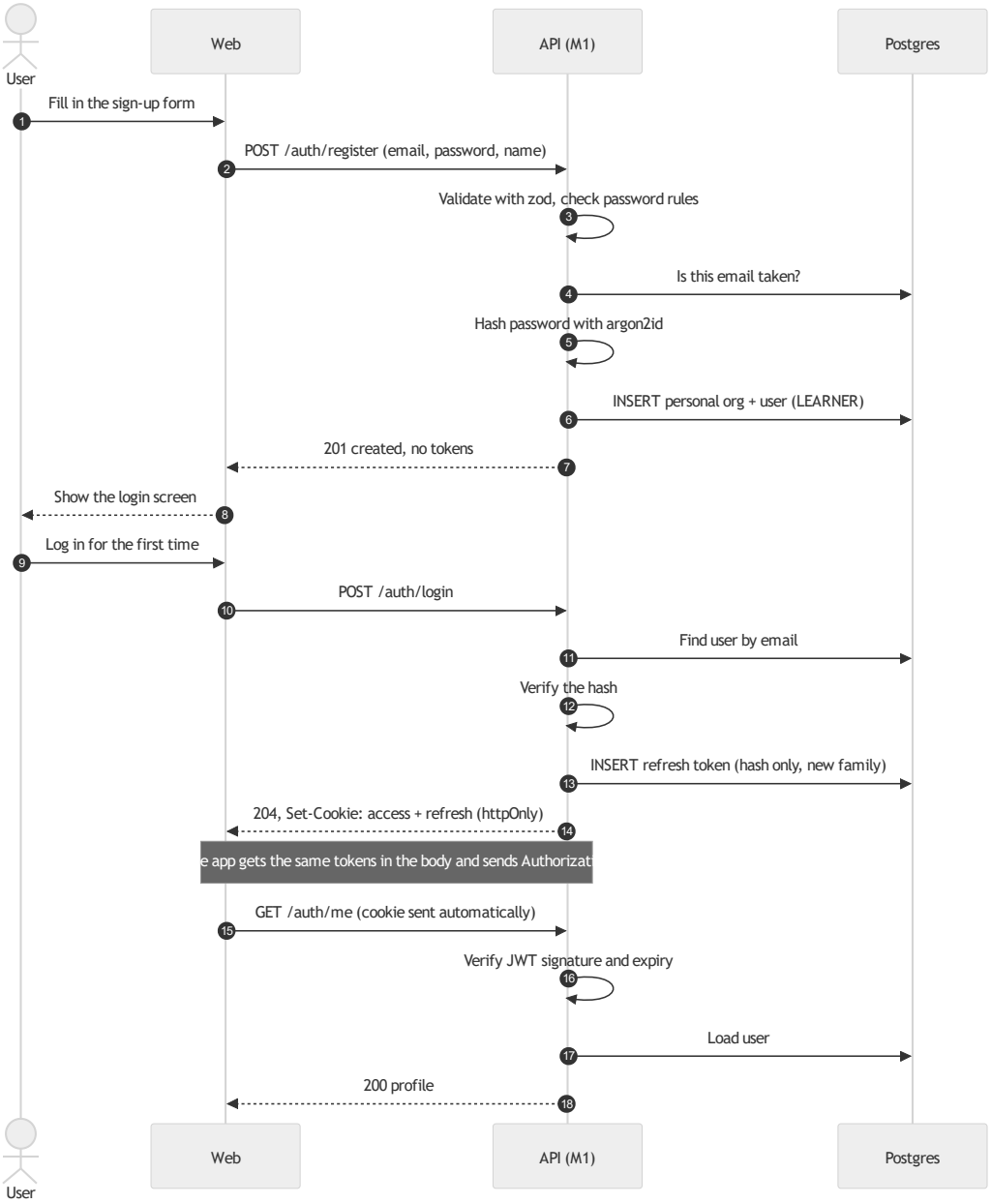
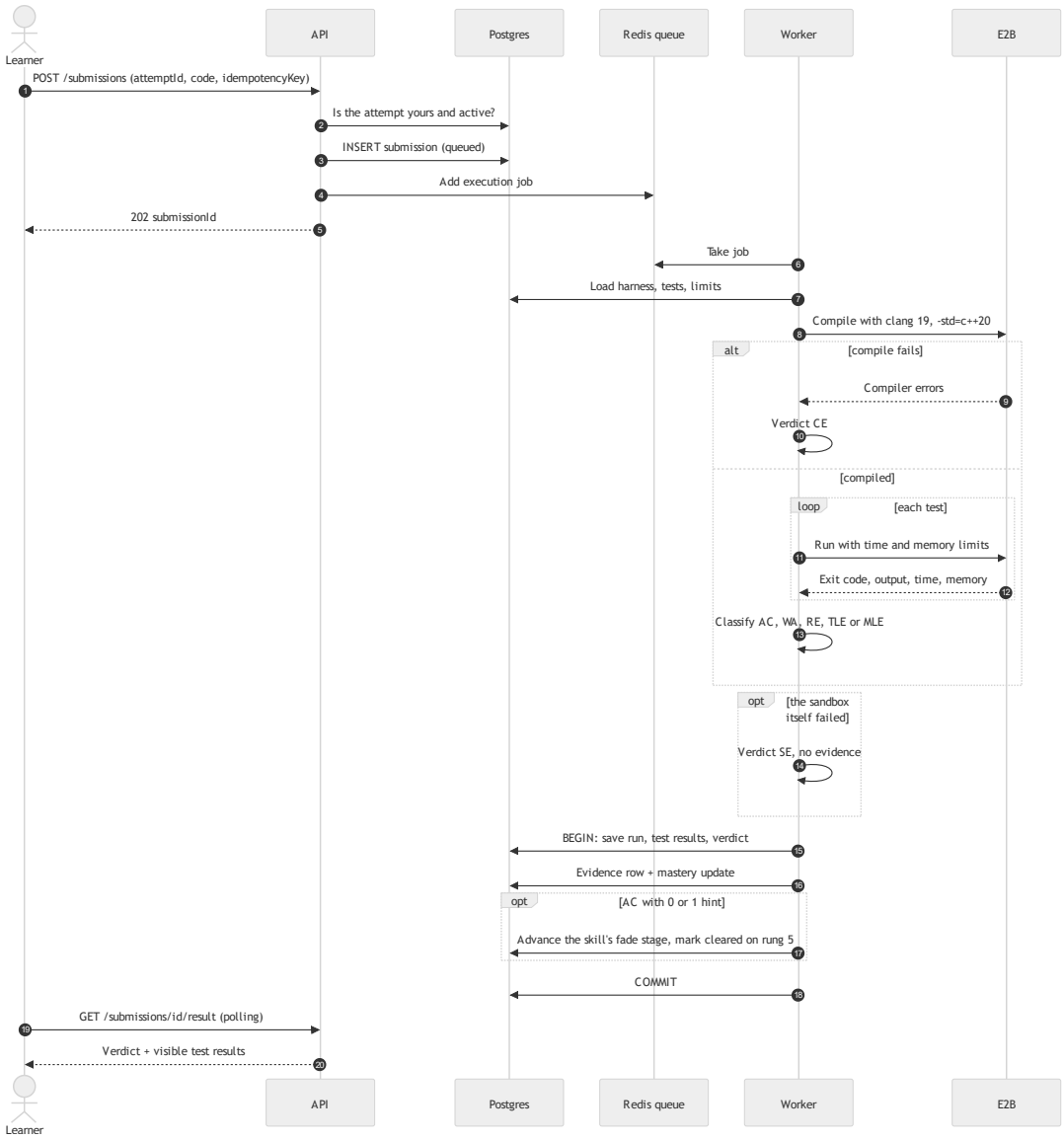


Figure 6: Submit code. This is the central flow of the system. The API saves the submission and answers `202` at once; a worker grades it in a fresh sandbox and saves the verdict, the mastery evidence and any fade advance in **one database transaction**.



3.2.3 Activity and State Diagrams

Figure 7: State of a submission. A sandbox failure is retried with a fresh sandbox; after the last try the verdict is SE, which never counts against the learner.

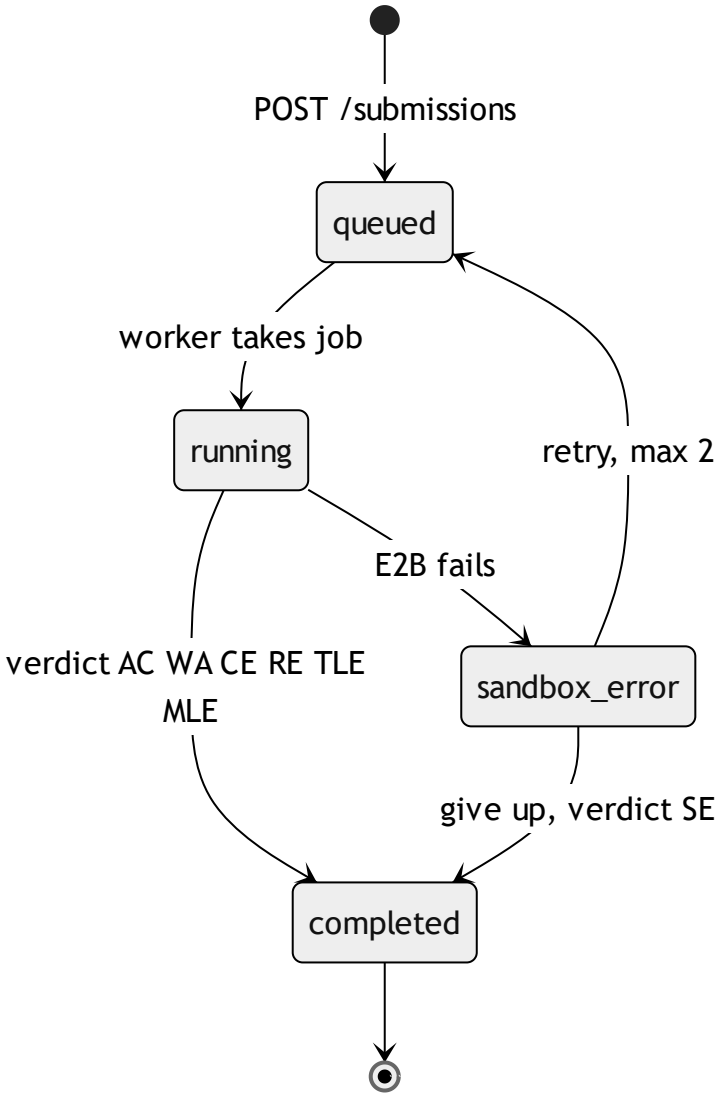


Figure 8: Activity diagram, next-step selection, as built (recommender m5-v1, ADR-0011). No LLM is involved; every pick is saved with its reasons.

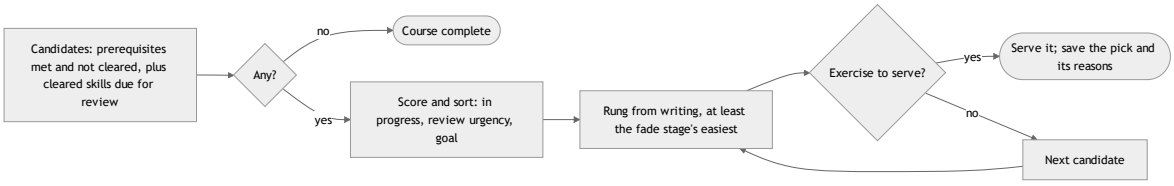


Figure 14: Request a hint, as built (ADR-0015). One level per request, never above the fade stage's ceiling. The live tutor comes first; the frozen hint is the fallback.

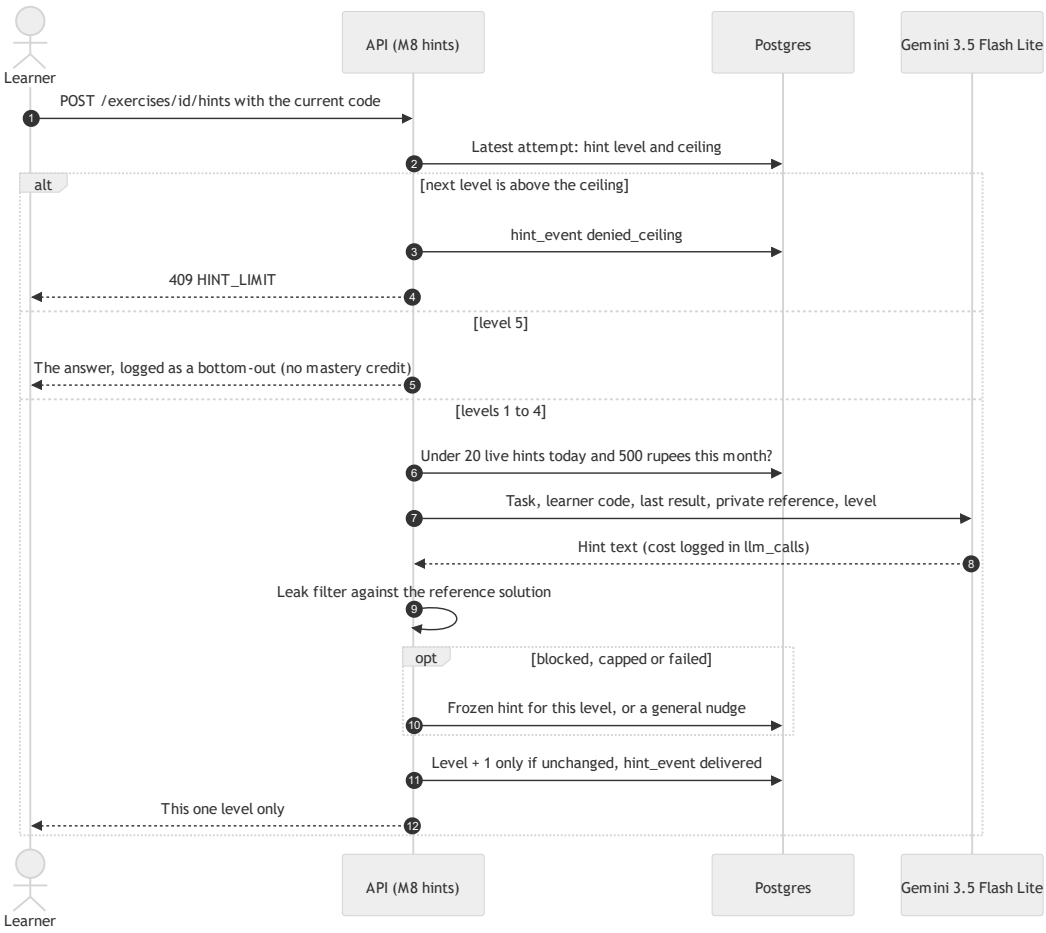
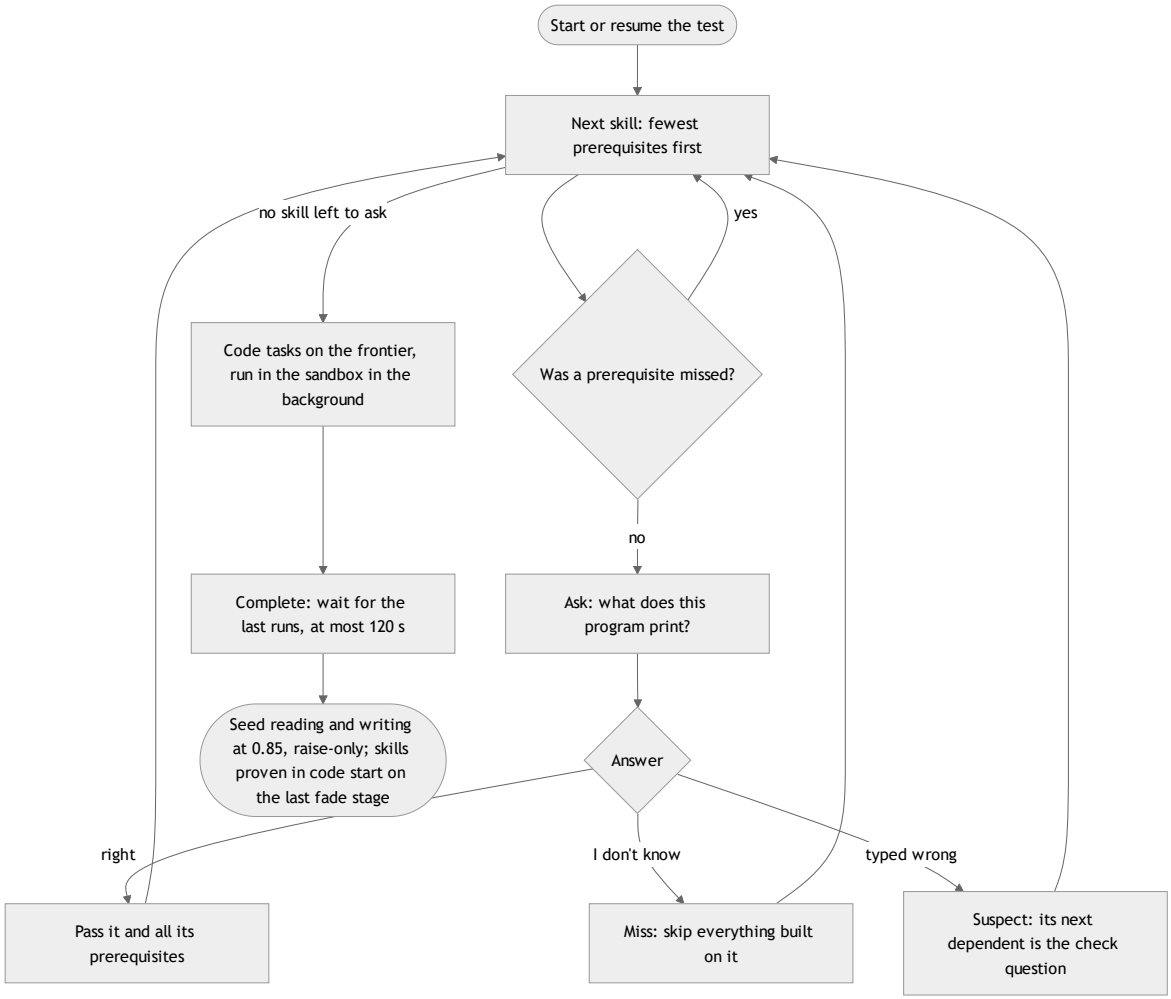


Figure 15: Placement test, as built (ADR-0012). Reading questions walk the skill graph easiest-first; code tasks are asked only on the frontier of what was read correctly.



3.2.4 Class Diagrams

Figure 3: Domain model

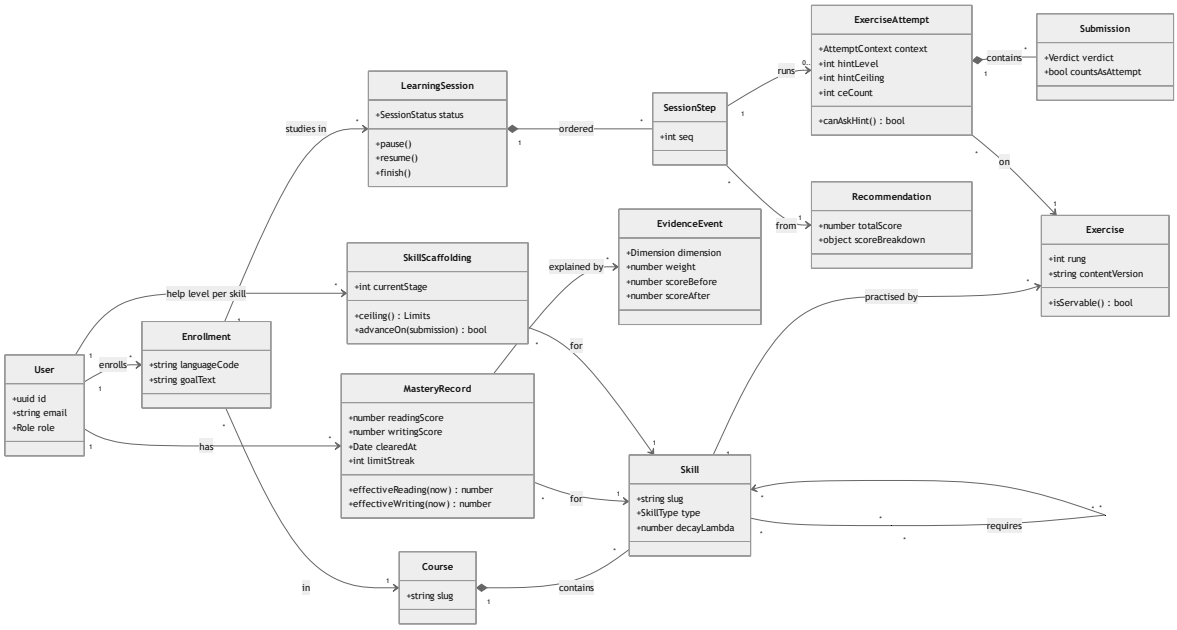
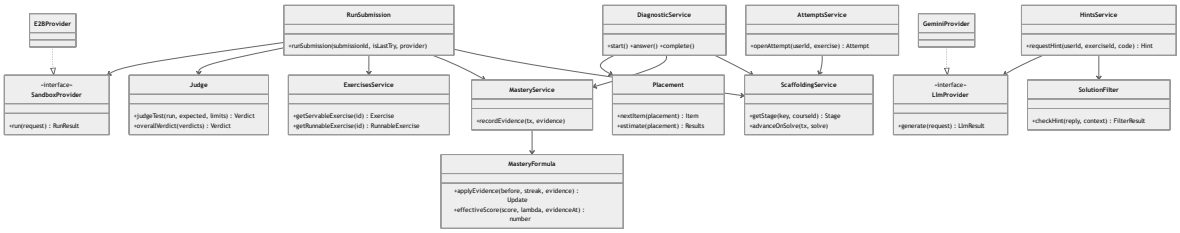


Figure 4: Services and adapters, as built. Two interfaces keep the outside services replaceable: `SandboxProvider` (only one class knows about E2B) and `LlmProvider` (only one class knows about Gemini). Tests replace both with fakes. Modules call each other's services, never each other's tables.



3.3 Database Design

Design rules applied:

- **Ledger plus snapshot.** Every mastery change appends an `evidence_events` row that is never edited; `mastery_records` holds today's value. Both are written in one transaction, so the history always explains the number.
- **Compute on read.** Effective mastery (after forgetting) and "review is due" are calculated when read, never stored, because they change every second.
- **Frozen content.** An exercise is identified by its content key (skill, language, rung, variant, content version). Its secret parts (harness, reference solution) live in a separate table, `exercise_secrets`, that no learner-facing query touches.
- **Normalisation.** Tables are in third normal form: each non-key column depends only on its table's key. There are two deliberate, documented exceptions. `submissions.user_id` is copied from the attempt to make per-learner queries simple. `mastery_records` is a snapshot of the ledger, kept consistent by always writing both in one transaction.

Validations enforced by the database itself:

Rule	How it is enforced
Scores between 0 and 1; rungs 1–5; hint level 0–5	CHECK constraints (`inRange` helper)
A hint level never above the attempt's ceiling	CHECK `hint_level <= hint_ceiling`
One active attempt per learner and exercise	Partial unique index `WHERE status = 'active'`
A submission never runs twice	UNIQUE `idempotency_key`
Evidence never counted twice	UNIQUE `(submission_id, dimension)`

Rule	How it is enforced
One live consent of each kind	Partial unique index `WHERE revoked_at IS NULL`
Email unique in any letter case	`citext` column type
A skill can only sit on a stage its schedule has	Composite foreign key

The schema has **42 tables in 9 groups**: 38 are built (13 migrations), and 4 come later with their features.

3.3.1 ERD

Figure 9a: Overview of the table groups

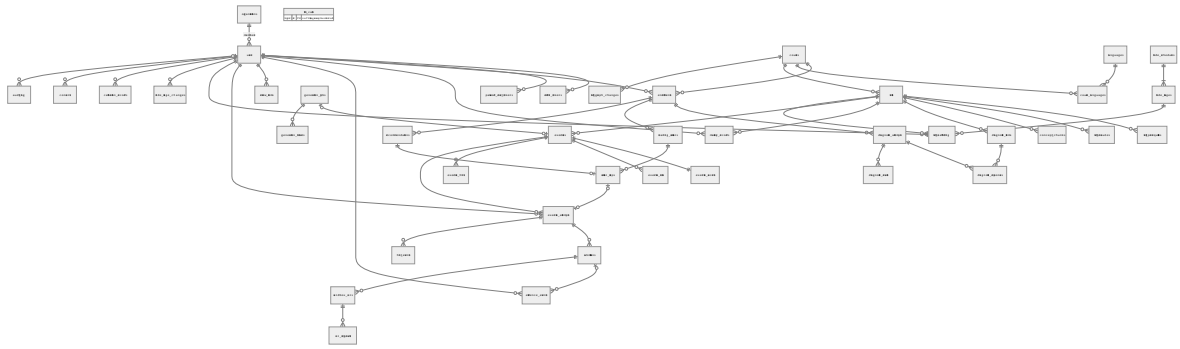
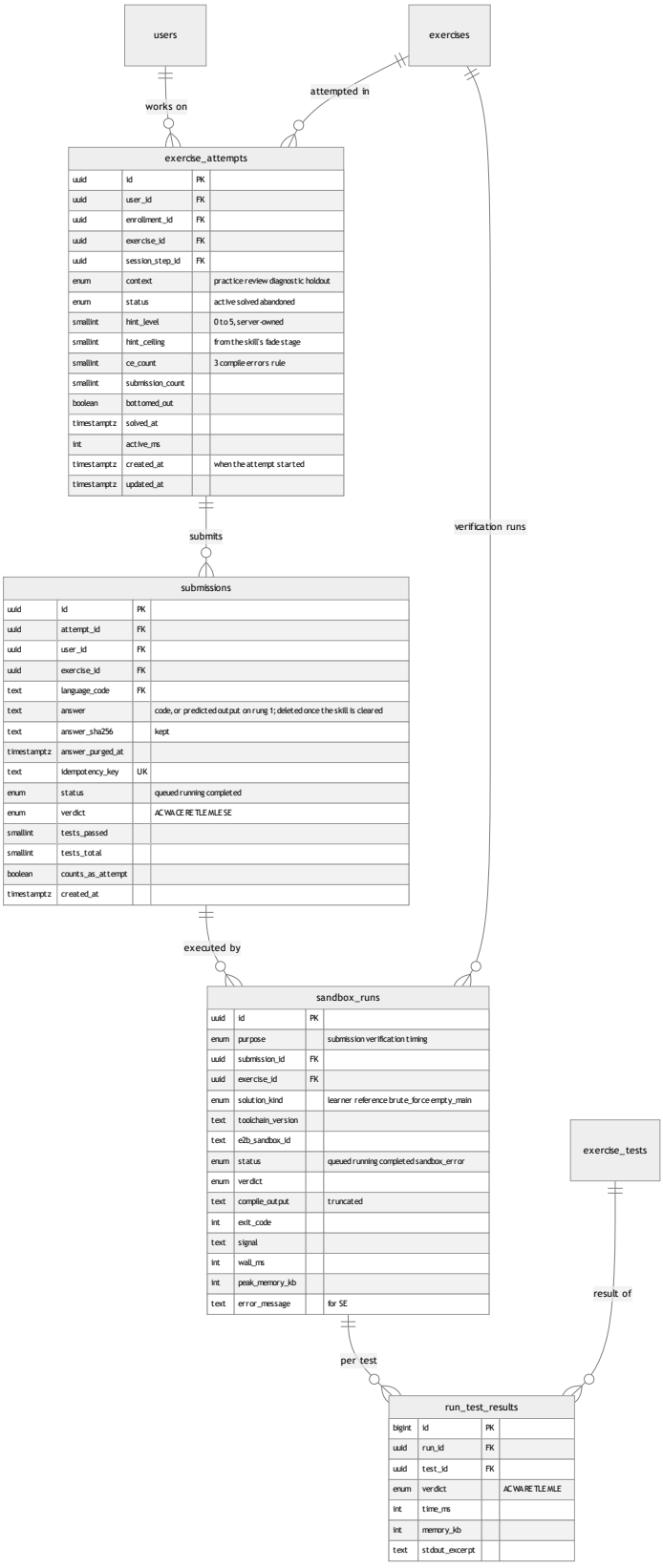


Figure 9b: Attempts and execution, the tables used by the submit flow



3.3.2 DFD

Figure 10a: Level 0 (context)

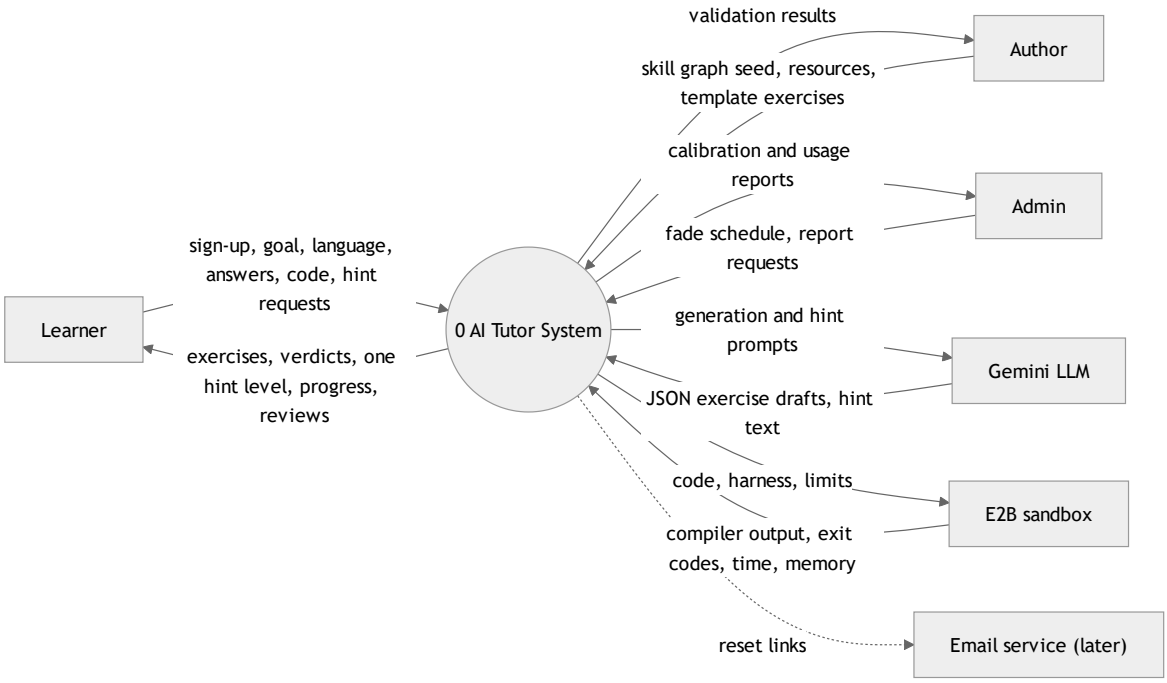


Figure 10b: Level 1 (the 13 processes)

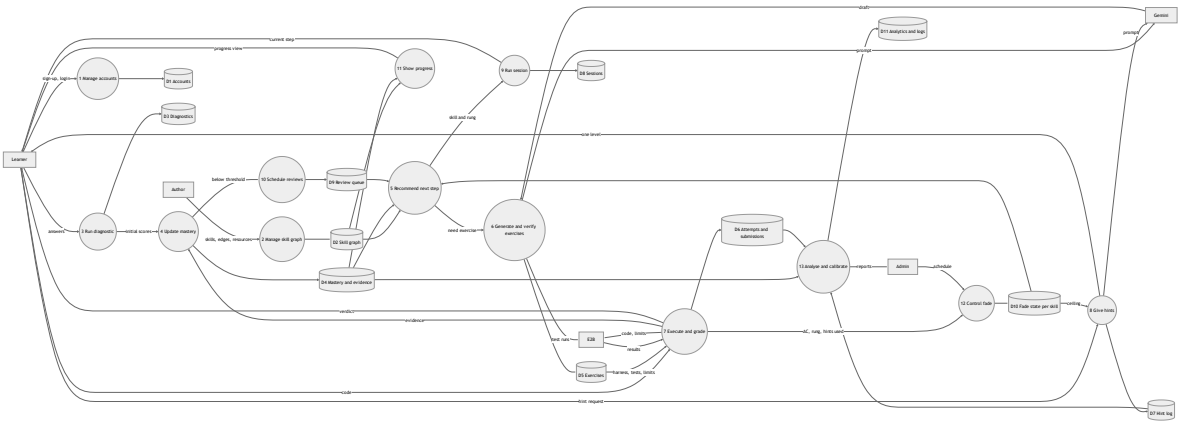
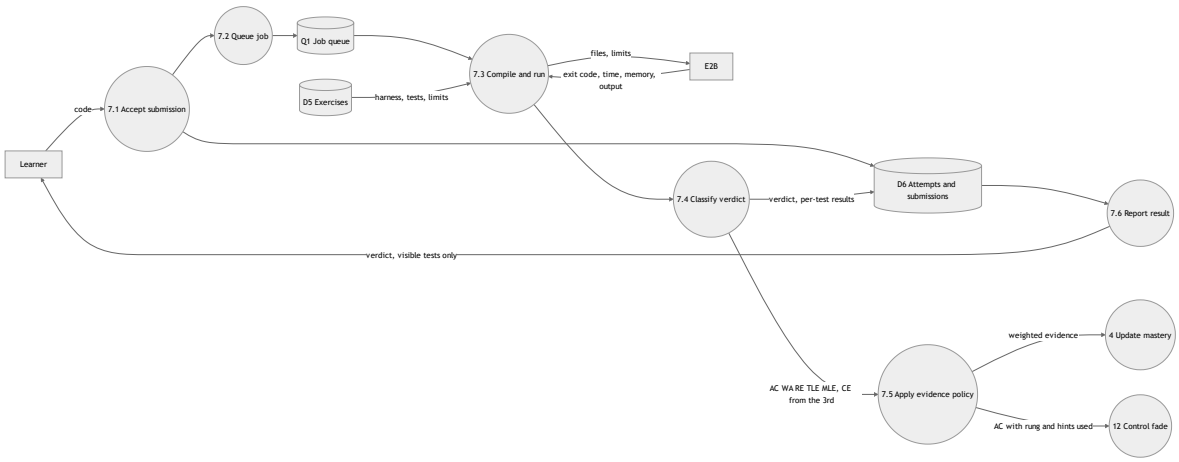


Figure 10c: Level 2, process 7 (execute and grade)



3.4 Input Design

All input enters through the REST API, and **every request body is validated by a zod schema before any logic runs**. Invalid input returns `400 VALIDATION_ERROR` with a message per field.

Input	Fields	Validation
Register	email, password, displayName, dateOfBirth, acceptTerms, acceptPrivacy, aiTraining?, research?	Email ≤ 254 characters, case-insensitive; password 8–128 characters (NIST: no forced symbols); name 1–80; date of birth a real date, not in the future; both acceptances must be `true`; opt-ins refused for under-18s; unknown fields such as `role` are stripped
Login	email, password, client (web or mobile)	Same email rules; failed logins throttled per email and per IP
Submit code	attemptId, answer, idempotencyKey	attemptId must be a UUID of *your own* active attempt; answer ≤ 64 KB; key 8–100 characters; 10 submits per minute per learner
Read mastery / fade state / progress	`language` (and `course`) query parameters	Optional, 1–20 (60) characters; the course must be offered in that language
Placement answer	itemId, then answer or dontKnow	Only the current question is accepted (`409 ITEM_NOT_CURRENT` otherwise); answer ≤ 64 KB with no NUL character; 30 calls per minute
Session change	status: active, paused or ended	An ended session refuses changes
Hint request	code (optional)	≤ 64 KB; the level is decided by the server, never the client; 10 per minute
Analytics	days (1–365), course, language	Admin role only (`403 FORBIDDEN` otherwise)

Screen input (frontend, in progress): sign-up form with a date picker and two separate required tick boxes, with the optional opt-ins shown only when the date of birth makes the user 18+; login form; exercise page with the Monaco editor (a plain text box for rung-1 "predict the output"). The same rules are applied in the browser to show errors before submitting, but the server is always the authority.

3.5 Output Design

Every response has **one envelope**:

```
{ "success": true,  "data": { } }
{  "success":  false,  "error":  {  "code":  "ATTEMPT_CLOSED",  "message":
"This attempt is already finished" } }
```

The client branches on `error.code` and shows `error.message`.

The submission result shows a sample test in full, and a hidden test as its verdict only:

```
{ "verdict": "WA", "testsPassed": 1, "testsTotal": 4,
  "tests": [
     { "seq": 1, "name": "Example", "visibility": "sample", "verdict":
"WA",
      "input": "123", "expectedOutput": "6", "output": "3\n" },
      { "seq": 2, "name": "Zero", "visibility": "hidden", "verdict":
"AC", "timeMs": 0 } ] }
```

A placement question carries the program or the task, never its answer key, its tests or whether earlier answers were right:

```
{   "itemId":   "…",   "dimension":   "reading",   "skill":   {   "slug":
"operators", "name": "Operators" },
   "promptMd": "Read the program carefully. **What does it print?**",
"code": "#include <iostream>\n…" }
```

A hint is one level only, with where it came from:

```
{ "level": 2, "ceiling": 5, "source": "live", "bodyMd": "Look at the
line that returns the result: which variable does it use?" }
```

Mastery output always shows reading and writing **separately**, each as a stored `score` and an `effective` score after forgetting. They are never blended into one number, because a 10/11 versus 0/4 profile averages into a misleading 65%.

3.6 Code Design and Development

```
apps/api/src/
  config/env.ts                 settings, validated at startup
  infrastructure/
    database/  schema/ seed/    Drizzle tables, migrations, seed data
    sandbox/                     SandboxProvider interface, E2B adapter,
template
    llm/                          LlmProvider interface, Gemini adapter,
cost log
    queue/                       BullMQ queue and worker
   shared/                              errors, response envelope, auth
helpers, middlewares, India time
  modules/                       each: routes → controllers → services →
repositories
      m1_auth   m2_skills   m3_diagnostic   m4_mastery   m5_recommender
```

```
m6_exercises
          m7_execution     m8_socratic     m9_sessions     m11_progress
m12_scaffolding  m13_analytics
  scripts/                       sandbox build and smoke test, exercise
verification, hint generation, make-admin
```

Key pieces of logic are pure functions, which means no database and no network, so they are fully unit-tested:

- `judge.ts`: turns sandbox readings into a verdict (Table 5).
- `mastery-formula.ts`: the M4 formula and decay (Table 6).
- `age.ts`: whole-year age, used for the under-18 rule.
- `recommend.ts`: the next-step rules (Figure 8).
- `placement.ts`: the placement walk and estimates (Table 11, Figure 15).
- `session-clock.ts`: idle pause after 15 minutes, end after 24 hours paused, study time.
- `solution-filter.ts`: blocks hints that resemble the solution (Table 12).
- `exercise-draft.ts`: what the model must write for a new exercise, the prompt, and the conversion into the hand-written bank's format (Table 13).

Table 5: Verdict rules. These are checked in this order for each test; the first failing test decides.

Verdict	Rule
CE	The compiler exits non-zero, or takes longer than 10 s
TLE	The test was killed by the time limit (2 s by default)
MLE	Peak memory above the limit (256 MB), or a `std::bad_alloc` crash
RE	Any other non-zero exit (crash, divide by zero)
WA	Output differs from the expected output (trailing spaces and line endings ignored)
AC	All tests pass
SE	The sandbox itself failed; retried, never counted

Table 6: Mastery formula v1. Rung 1 moves reading; rungs 2–5 move writing.

Event	Update
AC	new = old + 0.40 × weight × (1 − old); weight = rung/5 × (1 − 0.15 per hint; 0 if the solution was shown) × time factor (0.7–1)
WA or RE	new = old × (1 − 0.15 × (6 − rung)/5); **only the first failure in an attempt** is penalised

Event	Update
TLE or MLE	Penalty starts at 1/20 of a WA, doubles on each consecutive one, capped at a WA; only an AC resets it
Compile error, sandbox error	No change
Decay (on read)	effective = score × e^(−λ × days since evidence)

About five good solves (rungs 2 → 5) take a new skill from 0 to 0.72, just past the 0.7 prerequisite threshold.

Table 7: Fade stages. Help is withdrawn per skill.

Stage	Easiest rung	Max hints	Moves on after solving
1	1	5	rung 2 (or harder) with ≤ 1 hint
2	3	4	rung 3 with ≤ 1 hint
3	4	3	rung 4 with ≤ 1 hint
4	5	2	rung 5 with ≤ 1 hint → skill cleared

Table 11: Placement test rules (ADR-0012). All 20 skills are covered by 40 frozen, sandbox-verified items: one predict-the-output reading item and one small function per skill.

Rule	Detail
Order	Fewest transitive prerequisites first, then difficulty, then name
Reading walk	A skill is asked only if no prerequisite was missed. A right answer passes the skill and all its prerequisites
Slip check	"I don't know" is a miss. A typed wrong answer is only *suspect* until its first dependent is answered: right rescues it, wrong confirms both
Code tasks	Only on the frontier (passed skills nothing else passed builds on); an AC vouches for everything below. Run in the background
Estimate	Passed → 0.85 for reading; proven in code → 0.85 for writing; confidence 0, 0.5 or 1 by what was observed
Seeding	One transaction: mastery raised only if higher than today's value; skills proven in code start on the last fade stage; skills cleared in practice untouched
Length	1 question for a novice, 6 for a beginner, 18 for a mid learner, 26 (about 28 minutes) for an expert, instead of 40
Retakes	Up to 3 per course; a retake can lift a placement but never lowers it

Table 12: Hint ladder and where each hint comes from (ADR-0015).

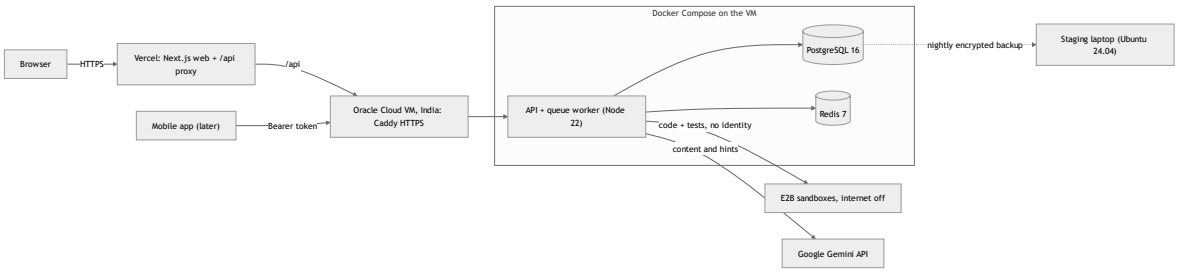
Level	Content	Source
1 Concept	The idea or C++ feature needed, in words	Live tutor first; frozen hint if capped, failed or blocked
2 Location	Where in the learner's own code the problem is	Same
3 Mechanism	How to fix it, in words	Same
4 Worked example	A different, smaller problem using the same technique	Same
5 Answer	The reference solution, or the program's output for rung 1	Always frozen; logged as a bottom-out; the attempt earns no credit

Table 13: Exercise generation: how a job runs (ADR-0016). Started by an admin or author, or queued automatically by the recommender when a learner's best next skill has no exercise.

Step	Rule
Queue	One active job per skill and rung; a second request gets the same job. Run one at a time in the background
Tries	Up to 3: Gemini 3.8 Flash, Flash, then Gemini 3.1 Pro. Each retry is told why the last draft was rejected
Draft	JSON checked against a schema: title, prompt, function signature, the call each test makes, starter, reference, 3–6 test cases, and 3 wrong solutions (rung 1: a program and its claimed output)
Prompt	The skill, the only other skills it may use (its prerequisites), the rung's rule, the frozen harness, an example from the hand-written bank, and the titles to avoid
Proof	In the real sandbox: the reference passes every test, the starter compiles but fails one, and each wrong solution compiles but fails one. A rung-1 program must print exactly the output it claims
Store	Only proven content is stored, ready to serve; its frozen hints follow
Failure	Every failed try is logged with its stage. After 3 tries, or when this month's ₹300 is spent, the job falls back to the hand-written exercise for that skill and rung

A live hint is blocked if it holds 25 % or more of the code only the solution has (5-token fingerprints, not counting the starter or the learner's own code), shows a code block longer than 6 lines, or gives away a rung-1 output.

Figure 12: Deployment, as decided for the MVP (ADR-0010)


Browser
HTTPS
Vercel: Next.js web + /api proxy
/api
Mobile app (later)
Bearer token
Oracle Cloud VM, India: Caddy HTTPS
Docker Compose on the VM
API + queue worker (Node 22)
PostgreSQL 16
Redis 7
nightly encrypted backup
Staging laptop (Ubuntu 24.04)
code + tests, no identity
content and hints
E2B sandboxes, internet off
Google Gemini API

CHAPTER 4: TESTING AND IMPLEMENTATION

4.1 Testing Methodology

Tests are automated with **Vitest** and **Supertest** and run against a **separate test database** (`ai_tutor_test`), which is rebuilt from the migrations on every run. That way the tests prove the real migrations, not a hand-made copy. Each test creates its own users and data with random names, so tests never depend on each other.

4.1.1 Unit Testing

Pure functions are tested with constructed inputs:

- **judge:** each verdict rule, rule order (TLE wins over a crash exit code), output normalisation, and an empty test list giving SE, never AC;
- **mastery formula:** the approved learner journey to the exact numbers, the hint and solution-shown rules, TLE doubling and its cap, and decay to half after $\ln 2 / \lambda$ days;
- **age:** turning 18 exactly on the birthday;
- **skill graph:** cycle detection, plus a check that the seeded C++ course has unique names, only known prerequisites and no cycles;
- **recommender:** candidates, review urgency, rung bands clamped to the fade stage, and the journey from rung 1 to rung 5;
- **placement:** every one of the **711 knowledge states** the 20-skill graph allows, answered with "I don't know" and with wrong guesses, is placed exactly, in at most 20 reading and 7 code questions; slips, missing items and cleared skills;
- **session clock:** the idle pause, the 24-hour end, and study time that never counts idle gaps;
- **leak filter:** prose and fragments pass; the solution, whole or in its key lines, fenced or not, is blocked; the learner's own code is not counted; a rung-1 output is never given away;
- **exercise drafts:** the schema's rules (the test call must take the test's arguments; exactly 3 wrong solutions), the conversion into the bank's format, and what the prompt contains.

4.1.2 Module Testing

Database-rule tests, one per schema group, try to break each constraint on purpose and expect the database to refuse. Examples: a duplicate email in different letter case, a hint level above the ceiling, a second active attempt, a second open session, a duplicate idempotency key, evidence with no source.

4.1.3 Integration Testing

HTTP-level tests call the real Express app with real requests:

- **auth:** register, login, refresh, logout, theft detection, throttling, CSRF and consent;
- **submissions:** submit, idempotency, ownership, judging, retries;
- **exercises:** secrets never leak; attempts, enrollment and fade start;
- **mastery and fade:** read views, the decay maths, stage advance;
- **recommender:** first step, moving up rungs, skipping skills without content, course complete;
- **placement test:** no answer key in any response, only the current question accepted, replays, code tasks queued once, the 202 grading wait, seeding, fast-track to the last stage, retakes, and a cheat attempt with a borrowed idempotency key;
- **sessions and progress:** steps from the recommender, pause/resume at the same step, idle and timeout, statuses that match the recommender's unlock rule;
- **analytics:** admin only, counts only, never a learner's email;
- **hints:** live hint, a leaky reply replaced by the frozen hint, a failed call, the fade ceiling, the answer at level 5, the daily and monthly caps;
- **exercise generation:** a proven draft stored ready to serve; a retry after a bad reply; tests a wrong answer passes, rejected three times, ending on the Pro model and the hand-written fallback; a rung-1 program's claimed output checked; the budget stop; admin-only endpoints; automatic queueing by the recommender.

The sandbox and Gemini are replaced by **fake providers** in these tests: same interfaces, fixed results. That makes them fast, free and repeatable.

4.1.4 System Testing

1. **Sandbox smoke test** (`pnpm sandbox:smoke`): six real C++ programs run on the real E2B service, and each must produce its intended verdict (AC, WA, CE, RE, TLE, MLE).

2. **Content verification** (`pnpm exercises:verify`): every seeded exercise runs in the real sandbox; the reference must pass and the starter must fail. **66 of 66 verified** (26 practice exercises, 40 placement items).
3. **End-to-end learner loop** on a running server: register → log in → open exercise → submit → poll → read mastery → reopen the exercise.
4. **End-to-end placement test** on real E2B, then the first recommendation and the fade stages; **sessions** (start, pause, resume, end); **progress**; **hints** (the whole ladder); **analytics** with a promoted admin; **generation** refused correctly while billing is off.
5. **Manual API tests** with curl, for authentication and consent, by a team member.

4.1.5 White-Box and Black-Box Testing

- **White box:** unit tests cover every branch of the judge and formula, and database-rule tests target specific constraints.
- **Black box:** integration and system tests use only the public API and check visible behaviour. For example, "a hidden test's input never appears in any response" is checked by searching the whole response body.

4.1.6 Acceptance Testing

The acceptance criteria are the SRS requirements, traced in the Phase 2 design document (section 9). For example:

- **FR-M4-02**, "mastery changes only from executed evidence": tested by "records no evidence for SE or an early compile error".
- **FR-M12-02**, "the help limit is enforced on the server": tested by "never lets the hint level pass the fade ceiling" and "refuses a level above the fade stage's ceiling, and logs it".
- **FR-M8-04**, "block any reply that looks like the solution": tested by "blocks a reply that gives the solution away and serves the frozen hint instead".
- **FR-M3-01**, "an adaptive test estimates reading and writing": tested by "places all 711 possible learners exactly".

Learner acceptance testing is planned for the pilot before the final report.

4.2 Test Data and Test Cases

Table 8: Automated test suites (171 tests in 35 files, all passing on 19 Oct 2026)

Level	Suite	Tests	What it proves
Unit	judge	7	Each verdict rule, order, normalisation, excerpts
Unit	mastery formula v1	8	Approved journey 0 → 0.16 → 0.33 → 0.53 → 0.72; hints; TLE doubling; decay
Unit	skill graph	4	Cycle detection; seeded C++ graph is valid
Unit	recommender rules	7	Candidates, reviews, rung bands, journey from rung 1 to 5
Unit	placement rules	11	All 711 knowledge states placed exactly; slip check; frontier code tasks
Unit	session clock	5	Idle pause, timeout end, study time
Unit	leak filter	5	Solution-like hints blocked, learner's own code ignored
Unit	exercise banks	6	Practice and placement banks complete and well-formed
Unit	exercise drafts	5	Draft rules, conversion to the bank's format, prompt contents
Module	9 schema suites	16	Every key constraint refuses bad data
Integration	auth + consent	14	Login flows, token theft, throttling, CSRF; DPDP consent and under-18 hold
Integration	submissions	10	202 + polling, idempotency tied to its attempt, hidden tests private, CE/SE rules
Integration	exercises API + verification	11	No secrets or rung-1 answers in responses; bad content rejected; wrong solutions must fail
Integration	mastery API, scaffolding fade	7	Reading and writing apart; decay on read; stage advance
Integration	recommendations API	4	First step, rung up, skip skills without content, course done
Integration	placement test API	18	No answer key, current question only, background grading, seeding, retakes, cheat refused
Integration	sessions, progress	8	Steps advance, pause/resume, idle, statuses
Integration	analytics	5	Admin only, counts only
Integration	hints, budget, frozen hints	10	Live, blocked, failed, ceiling, answer, daily and monthly caps

Level	Suite	Tests	What it proves
Integration	exercise generation, budget, API	10	Proven drafts stored; retries; weak tests rejected; Pro then fallback; budget stop; admin only; automatic queueing
System	sandbox smoke (real E2B)	6	All six execution verdicts on real infrastructure
System	content verification (real E2B)	66	Every seeded exercise: reference passes, starter fails

Sample test case: a generated exercise whose tests can't catch a mistake is rejected

Field	Value
Input	The (fake) model writes a correct exercise, but one of its 3 "wrong" solutions is actually correct, so it passes every test; it does this on all 3 tries
Expected	Each try is rejected at stage `tests_too_weak`; the third uses the Pro model; the job ends `fell_back`, pointing at the hand-written exercise of the same skill and rung; nothing is stored
Result	Pass

Sample test case: a hint never gives the solution away

Field	Value
Input	A learner at hint level 1 asks for level 2; the (fake) tutor replies with the whole reference solution in a code block
Expected	The reply is blocked and logged as `blocked`; the learner gets the frozen level-2 hint instead (`source: "frozen"`); the level moves to 2
Result	Pass

Sample test case: judging hides hidden tests

Field	Value
Input	Exercise with 1 sample test (input `123`, expected `6`) and 1 hidden test; the learner's run outputs `6` and `27`
Expected	Verdict AC, 2 of 2 passed; the sample test shows input, expected and actual output; **the hidden test shows only `{ seq, name, visibility, verdict, timeMs }`**
Result	Pass

Test data for the starter exercise "Sum of digits" (`int sumOfDigits(int n)`):

Test	Visibility	Input	Expected
Example	sample	123	6
Zero	hidden	0	0
All nines	hidden	999999999	81
Largest input	hidden	1000000000	1

4.3 Test Reports and Debugging

Results: 171 of 171 automated tests pass, the 6 of 6 real-sandbox verdicts are correct, 66 of 66 seeded exercises verify on the real sandbox, and the end-to-end runs pass (A-8, A-10).

Debugging followed one rule: **measure, find the cause, fix the cause, add a test.** These are the defects found:

Table 9: Defects found and fixed

#	Symptom	Cause found	Fix
1	The memory-hog test program got **AC** instead of MLE	The C++ optimiser legally removed an allocation that was never used	Test program now writes to the memory (`volatile`), so it can't be removed. Judge unchanged
2	A submission took **5–15 s**	Timing each step showed compiling took only 0.5 s; each network round trip to the sandbox cost 1–2 s	Compile + all tests in **one** command; template pre-warms the compiler; sandbox killed in the background. Now about 6 s
3	Any non-zero exit **crashed the runner**	The SDK throws an exception on a non-zero exit	The script always exits 0 and reports each exit code as data
4	Learner output could **break or forge** the result format	Results were read from text the program could also print	Each result line is written by our script *after* the program ends, with output base64-encoded
5	**Cheating hole:** printing "PASS" could pass	The first design let the harness inside the sandbox decide pass or fail	Expected outputs never enter the sandbox; the judge compares outside it
6	Migration 0011 **failed** on existing data	Shrinking an enum fails while rows still use a removed value	The migration deletes those rows first (it had never been applied anywhere, so editing it was safe)

#	Symptom	Cause found	Fix
7	Register returned **400** in manual tests	The JSON was quoted for PowerShell but run in Git Bash; one field was called `name` instead of `displayName`	Shell-specific commands in the guide; field names checked against the API
8	New tests failed with "`expect` is not a function"	An `async` helper returning a Supertest request was awaited automatically	The helper now takes the expected status itself
9	**Cheating hole:** a learner could be placed without writing code	A practice submission sent under the placement task's predictable idempotency key was returned as the placement answer	A key now returns only a submission made on the same attempt, otherwise `409` (this also protects practice)
10	A learner who had practised **once** lost their placement, and the skills built on it stayed locked	Seeding skipped any skill with a counted attempt	Only cleared skills are skipped; seeding is raise-only, so practice gains are never lowered
11	Risk of the API **freezing** under load	Inside the completion transaction, a skill lookup asked the pool for a second connection; with 10 such transactions at once, all would wait forever	The lookup now runs on the transaction's own connection (also fixed in grading)
12	The answer to a **rung-1** exercise was visible	Its only test was marked "sample", and sample tests are shown with the exercise	Rung-1 exercises never show sample tests
13	Git showed a source file as **binary**	A script wrote a raw NUL byte instead of the `\0` escape	Replaced with the escape; every changed file is now scanned for NUL bytes
14	Live hints would have sent learner code to Google's **free tier**	The Gemini key's project had not activated billing	A `GEMINI_PAID` setting must be `true` before any live call; until then hints fall back to frozen ones

4.4 Implementation Manual

Run the system locally:

```
docker compose up -d                  # PostgreSQL 16 + Redis 7
pnpm install
cd apps/api && cp .env.example .env    # fill DATABASE_URL, JWT_SECRET,
```

```
WEB_ORIGIN, E2B_API_KEY, GEMINI_API_KEY
pnpm db:migrate && pnpm db:seed        # 13 migrations; C++ course, 20
skills, 26 practice exercises, 40 placement items
pnpm exercises:verify                  # runs every new exercise in the
real sandbox before it can be served
pnpm dev                                    # API + queue worker on
http://localhost:3001
pnpm test                              # 171 automated tests
pnpm sandbox:smoke                       # real-sandbox check of all six
verdicts
pnpm user:admin <email>                    # make the first admin (for
analytics)
pnpm hints:generate                           # once billing is active
(GEMINI_PAID=true): freeze hints 1–4
```

Build the sandbox template once: `pnpm sandbox:build` creates the `ai-tutor-cpp` machine (Debian 13, clang 19) on E2B.

4.5 Implementation

Status at Review 3 (19 October 2026):

Module	Built	Endpoints live
M1 Accounts and DPDP consent	Register, login, refresh, logout, me; date of birth; consent kinds; under-18 hold	5
M2 Skill graph	C++ basics course: 20 skills, 23 prerequisites, further-reading links (seed)	–
M3 Placement test	Adaptive reading walk with a slip check, frontier code tasks, seeding, retakes; 40 verified items	3
M4 Mastery	Formula v1 inside the grading transaction; read views with decay; raise-only seeding	2
M5 Recommender	Next skill, rung and exercise by fixed rules, saved with reasons	1
M6 Exercises	Learner view; 26 verified practice exercises (first 5 skills × 5 rungs, plus one); generation proven in the sandbox, by admins and automatically	3
M7 Execution	Fresh offline sandbox per submission, judge, queue, retries, polling	2
M8 Hints	Five-level ladder, live tutor with leak filter and caps, frozen fallback	1
M9 Sessions	Start or resume, pause, resume, end; steps from the recommender; idle rules	3

Module	Built	Endpoints live
M11 Progress	Every skill, reading and writing apart, fade stage, status	1
M12 Fade	Stage start, advance on solve, fast-track from the placement test, state endpoint	1
M13 Analytics	Admin overview: learners, code runs, outcomes, content coverage, hardest exercises, hints, AI spend	1
Total		**all 23 MVP endpoints**

Real runs at this stage: a placement test on real E2B placed a learner who knew 3 skills with 6 questions (5 reading, 1 code), graded the code in about 6 seconds and fast-tracked those 3 skills to the last fade stage; the first recommendation was then the next new skill. Sessions, progress, hints, analytics and generation were each run against the development database (A-10).

Planned for the MVP (10 November): Gemini billing activation (7 November), then the first live AI runs: live hints, the frozen hints, and generated practice exercises for skills 6–20; frontend screens (in progress); deployment to the cloud server; and a pilot with real learners.

4.6 Users Training

- **Learners** need no training beyond the interface. Every verdict has a plain-language message ("Too slow on test 3: limit 2 s"), and the fade state tells the learner what unlocks the next stage.
- **The frontend team** works from a written API guide (`DOCS/api/FRONTEND-GUIDE.md`) with exact request and response types, error codes, and what to show for each verdict.
- **Authors** add skills and exercises as seed data; a test checks the skill graph for cycles on every run, and `pnpm exercises:verify` checks every new exercise in the real sandbox.
- **Admins** are created with `pnpm user:admin <email>` and see one overview page; it shows counts only, never who.

4.7 Post-Implementation Maintenance

- **Schema changes** always go into a new migration; applied migrations are never edited.
- **Decisions** are recorded as ADRs, so the reason for each choice survives the people who made it.
- **The mastery formula is versioned** (`m4-v1` is stored on every evidence row), so it can be tuned later from calibration data without making old numbers unexplainable.

- **Costs are watched:** every LLM call is logged with its estimated cost, with hard monthly caps; sandboxes are billed per second and killed after each run.
- **Data protection:** retention jobs (delete after 3 years inactive, with a 48-hour warning), a breach-response plan with the 6-hour CERT-In and 72-hour Board deadlines, and nightly backups.
- **Dependencies** are pinned to exact versions; upgrades are deliberate and tested.

CHAPTER 5: CONCLUSION AND REFERENCES

5.1 Conclusion

The project set out to measure what a learner can *write*, not what they have watched, and to withdraw help as they improve. At Review 3, the core of that idea works end to end on real infrastructure:

1. A learner opens an exercise, and an attempt starts with a help limit taken from the skill's fade stage.
2. The learner's C++ code is compiled and tested in a fresh, offline sandbox.
3. A deterministic judge gives the verdict.
4. In the same transaction, the writing score moves by the approved formula and the skill moves to less help when solved with at most one hint.

In a real run, a correct submission was judged **AC (4 of 4 tests) in about 6 seconds**, and the learner's writing mastery for that skill moved from **0 to 0.24**, exactly as the formula predicts.

Around that core, the rest of the learner's journey now works too: an adaptive **placement test** that asks an expert 26 questions instead of 40 and fast-tracks what they prove, a **recommender** that chooses the next skill, rung and exercise by explainable rules, **study sessions** that resume at the same step, an honest **progress** view, an **admin overview**, a **hint ladder** whose live tutor can never hand over the solution, and **exercise generation** that stores nothing the sandbox hasn't proven. All 23 MVP endpoints are live, backed by 171 automated tests, 66 exercises verified in the real sandbox, and a written decision record (ADR-0001 to ADR-0016) for every major choice.

5.2 System Specifications

5.2.1 H/W Requirement

- **Server:** 2+ CPU cores, 4 GB+ RAM, 20 GB disk (the Oracle Always Free VM offers up to 4 ARM cores and 24 GB).
- **Client:** any device with a modern browser.

5.2.2 S/W Requirement

- **Server:** Linux (Ubuntu 24.04), Docker; Node.js 22, PostgreSQL 16, Redis 7.
- **Accounts:** an E2B account (sandboxes) and a Gemini API key.
- **Client:** Chrome, Edge, Firefox or Safari (current versions).

5.3 Limitations of the System

- **No web interface yet.** The frontend is in progress; this draft's sample outputs are API responses, and screen snapshots will be added in the final report.
- **Practice content covers 5 of 20 skills.** The 26 practice exercises cover the first five skills (plus one); skills 6–20 get theirs from exercise generation, which is built and tested but first runs once billing is active. The placement test covers all 20.
- **Live AI waits for billing.** Live hints and exercise generation are built and tested with a fake model, but stay off until the ₹3,000 prepayment is complete and activates the paid tier (the remaining ₹2,500 on 7 November); until then learners get frozen hints or a general nudge. Lessons are not built yet.
- **The judge sees only output.** A determined learner can sometimes avoid a prerequisite (for example by rewriting a struct to use a vector); the tasks are designed so the natural solution uses it.
- **Placement length.** An expert answers 26 questions (about 28 minutes); a placement seed unlocks the next skills for about 4 days (syntactic skills) before one Build per skill confirms them.
- **Latency:** about 6 s per submission, mostly network round trips to the sandbox provider, whose servers are outside India.
- **Sandbox trust:** learner code runs as the same user as our runner script, so a determined learner could tamper with result lines. That's acceptable for practice; the fix is a separate run user.
- **Formula constants are starting values.** They will be calibrated against held-out exercises once real learners use the system.
- **Pilot:** the planned manual "Phase 0" validation was replaced by a single-learner baseline.

5.4 Future Scope for Modification

- More languages (Java, Python) on the same language-independent engine, and more courses (DSA, frontend, backend).

- The parent-approval flow for under-18 learners, with identity verification as the DPDP Rules require.
- Calibration of the mastery model from real data (held-out exercises), and spaced-revision scheduling (M10).
- Frozen lessons with a diagram and a worked example per skill, and the brute-force/time-limit check for DSA-sized generated exercises.
- A mobile app (the API already supports Bearer tokens), and colleges as institutional customers.
- Spoken explanation assessment, and a portable learner profile.

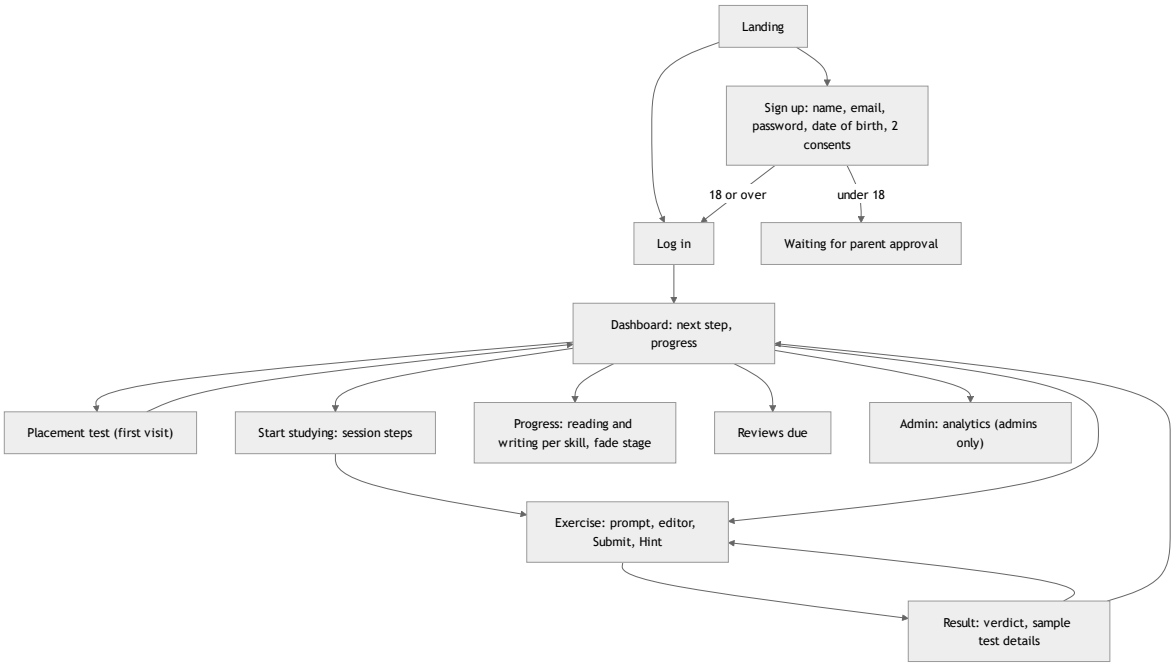
5.5 References / Bibliography

1. Bloom, B. S. (1984). The 2 Sigma Problem. *Educational Researcher*, 13(6), 4–16.
2. Roediger, H. L., & Karpicke, J. D. (2006). The Power of Testing Memory. *Perspectives on Psychological Science*, 1(3), 181–210.
3. Bjork, E. L., & Bjork, R. A. (2011). Making Things Hard on Yourself, But in a Good Way. In *Psychology and the Real World*, 56–64.
4. Cepeda, N. J., Pashler, H., Vul, E., Wixted, J. T., & Rohrer, D. (2006). Distributed Practice in Verbal Recall Tasks. *Psychological Bulletin*, 132(3), 354–380.
5. Sweller, J., & Cooper, G. A. (1985). The Use of Worked Examples as a Substitute for Problem Solving in Learning Algebra. *Cognition and Instruction*, 2(1), 59–89.
6. Corbett, A. T., & Anderson, J. R. (1994). Knowledge Tracing. *User Modeling and User-Adapted Interaction*, 4(4), 253–278.
7. VanLehn, K. (2011). The Relative Effectiveness of Human Tutoring, Intelligent Tutoring Systems, and Other Tutoring Systems. *Educational Psychologist*, 46(4), 197–221.
8. Bastani, H., et al. (2025). Generative AI Without Guardrails Can Harm Learning. *Proceedings of the National Academy of Sciences*.
9. Government of India (2023). *The Digital Personal Data Protection Act, 2023*; and (2025) *Digital Personal Data Protection Rules, 2025*.
10. Technical documentation: Node.js, Express 5, PostgreSQL 16, Drizzle ORM, BullMQ, E2B, Google Gemini API, Next.js (official documentation, accessed October 2026).
11. Project documents: Phase 1 synopsis, problem analysis, literature survey and SRS (24 Sep 2026); Phase 2 system design (15 Oct 2026, updated 19 Oct); ADR-0001 to ADR-0016; frontend API guide.

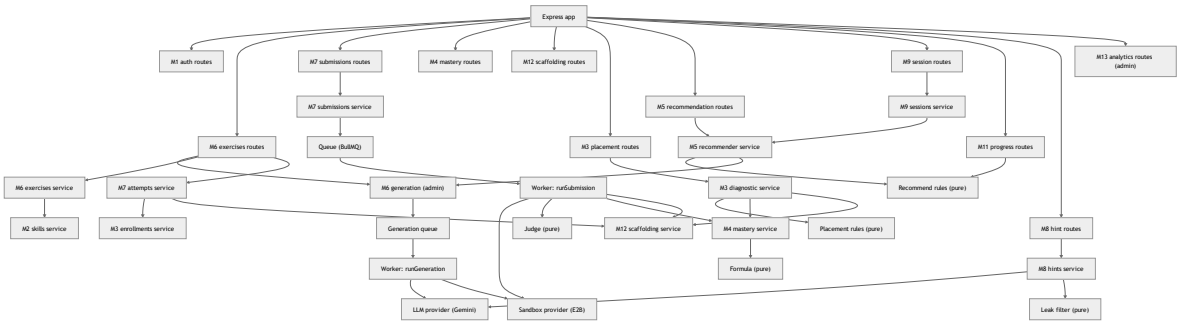
CHAPTER 6: ANNEXURES

A-1 Menu Flow Diagram

Figure 13: Planned screen flow



A-2 Structure Chart



A-6 Decision Tables

Sign-up consent

Age ≥ 18?	Accepted Terms and Privacy?	Opt-in ticked?	Result
any	No	any	400, sign-up refused
Yes	Yes	No	Active account; consents: data processing, terms
Yes	Yes	Yes	Active account; also the chosen opt-ins
No	Yes	No	Account held `pending_parent`; **no consent recorded**
No	Yes	Yes	400: opt-ins are for adults only

Does a submission count, and what does it do?

Verdict	Counts as attempt	Mastery	Fade
AC	Yes	Gain (weighted)	Advances if the rung and hints qualify
WA / RE	Yes	Penalty, only the first in the attempt	–
TLE / MLE	Yes	Small penalty, doubling	–
CE (1st, 2nd)	No	–	–
CE (3rd+)	Yes	No change	–
SE	No	–	–

Which hint does the learner get? (levels 1–4; level 5 is always the answer)

Above the ceiling?	Under the daily and monthly caps, billing on?	Gemini answered?	Filter passed?	Result
Yes	any	–	–	`409 HINT_LIMIT`, logged `denied_ceiling`
No	Yes	Yes	Yes	Live hint, logged `delivered` with its cost
No	Yes	Yes	No	Logged `blocked`; frozen hint (or general nudge)
No	Yes	No (error)	–	Frozen hint (or general nudge)

Above the ceiling?	Under the daily and monthly caps, billing on?	Gemini answered?	Filter passed?	Result
No	No	–	–	Frozen hint (or general nudge), no call made

Is a generated draft stored?

Valid JSON matching the schema?	Reference passes every test?	Starter compiles and fails?	Every wrong solution compiles and fails?	Result
No	–	–	–	Logged `schema`; next try
Yes	No	–	–	Logged `compile` or `reference`; next try
Yes	Yes	No	–	Logged `compile` or `tests_too_weak`; next try
Yes	Yes	Yes	No	Logged `compile` or `tests_too_weak`; next try
Yes	Yes	Yes	Yes	**Stored, ready to serve**; frozen hints follow

After the third failed try (or when the month's budget is spent), the job falls back to the hand-written exercise.

A-7 Data Dictionary

Table 10: Key tables (all 42 are described in the Phase 2 design document)

Table	Key columns	Constraints
users	id (UUIDv7), org_id, email (citext), display_name, password_hash (argon2id), role, status (active, disabled, pending_parent), date_of_birth	email unique in any case; name 1–80 characters
consents	user_id, kind (data_processing, terms, parental, ai_training, research_study), policy_version, granted_at, revoked_at	one live consent per kind

Table	Key columns	Constraints
exercises	id, skill_id, language_code, rung, variant_seed, content_version, status, title, prompt_md, starter_code, time_limit_ms, memory_limit_mb	content key unique; rung 1–5; limits in range
exercise_secrets	exercise_id, harness_code, reference_solution	never read by learner-facing queries
exercise_attempts	id, user_id, exercise_id, status, hint_level, hint_ceiling, ce_count, submission_count	one active per learner and exercise; hint_level ≤ hint_ceiling
submissions	id, attempt_id, answer (≤ 64 KB), answer_sha256, idempotency_key, status, verdict, tests_passed, counts_as_attempt	idempotency key unique
sandbox_runs / run_test_results	run status, verdict, compile output, wall time, peak memory; per-test verdict, time, memory, output excerpt	one result per run and test
mastery_records	(user_id, skill_id, language_code), reading_score, writing_score, evidence times, cleared_at, limit_streak, decay_lambda	scores 0–1; row locked while updated
evidence_events	user, skill, dimension, submission_id, verdict, rung, hints_used, weight, score_before, score_after, formula_version	exactly one source; never counted twice
skill_scaffolding / fade_stage_changes	current stage per learner and skill; every move with its cause (solved, or the placement test)	stage must exist in its schedule
diagnostic_items / attempts / responses / results	item (skill, reading or writing, verified exercise); attempt number and status; each answer; one estimate per skill	one test in progress per enrollment; one answer per item
recommendations	skill, rung, reason, score breakdown, top-5 snapshot, algorithm version, exercise served	rung 1–5
learning_sessions / session_steps	status, last activity, study time; ordered steps from recommendations	one open session per learner; one active step per session
generation_jobs / generation_failures	skill, rung, variant, trigger (admin or automatic), status, tries, model, the exercise made or the fallback; each failed try with its stage and reason	one active job per skill and rung; tries 0–3

Table	Key columns	Constraints
hint_events / llm_calls	each hint request (delivered, blocked, refused, bottomed out) with its text or stored hint; each Gemini call with tokens and estimated cost	hint level 1–5

A-8 Test Reports

Full test run of 19 October 2026: **35 test files, 171 tests, 171 passed.**

- ✓ analytics-api.test.ts > GET /analytics/overview > is for admins only
- ✓ analytics-api.test.ts > GET /analytics/overview > counts new learners, opt-ins, code runs, sessions and clears in the window
- ✓ analytics-api.test.ts > GET /analytics/overview > reports one course's content coverage and per-skill averages exactly
- ✓ analytics-api.test.ts > GET /analytics/overview > lists the hardest practice exercises, lowest solve rate first
- ✓ analytics-api.test.ts > GET /analytics/overview > accepts a window of 1 to 365 days
- ✓ attempts.test.ts > attempts schema > rejects a double-submit with the same idempotency key
- ✓ attempts.test.ts > attempts schema > never lets the hint level pass the fade ceiling
- ✓ auth.test.ts > auth > registers, logs in with cookies, and returns the profile without secrets
- ✓ auth.test.ts > auth > ignores a role sent at sign-up
- ✓ auth.test.ts > auth > rejects the same email in any letter case with 409
- ✓ auth.test.ts > auth > answers a wrong password and an unknown email identically
- ✓ auth.test.ts > auth > rotates refresh tokens, and revokes the whole login when an old one is reused
- ✓ auth.test.ts > auth > ends the session on logout
- ✓ auth.test.ts > auth > blocks cookie-carrying writes from another site
- ✓ auth.test.ts > auth > throttles the 6th failed login for one email
- ✓ auth.test.ts > consent at sign-up > records the required consents plus only the opt-ins chosen
- ✓ auth.test.ts > consent at sign-up > refuses sign-up without accepting the Terms

- ✓ auth.test.ts > consent at sign-up > refuses a date of birth in the future
- ✓ auth.test.ts > consent at sign-up > holds a minor for parental approval, with no consent recorded
- ✓ auth.test.ts > consent at sign-up > refuses AI-training or research opt-ins from a minor
- ✓ auth.test.ts > consent at sign-up > counts age in whole years, turning 18 on the birthday
- ✓ curriculum.test.ts > curriculum schema > rejects a skill that requires itself
- ✓ curriculum.test.ts > curriculum schema > rejects a duplicate edge even when it applies to every language (null)
- ✓ diagnostic-api.test.ts > POST /diagnostic/start > starts with the first skill's program to read, never its answer, and resumes the same test
- ✓ diagnostic-api.test.ts > POST /diagnostic/start > is unavailable until items are verified, and 404 for an unknown course
- ✓ diagnostic-api.test.ts > POST /diagnostic/:id/answers > grades reading on the spot without saying so, and accepts only the current question
- ✓ diagnostic-api.test.ts > POST /diagnostic/:id/answers > rejects an answer Postgres can't store as a 400, not a 500
- ✓ diagnostic-api.test.ts > POST /diagnostic/:id/answers > re-queues a code task whose first queueing failed when the answer is retried
- ✓ diagnostic-api.test.ts > POST /diagnostic/:id/answers > ends after an "I don't know" on the first skill, with nothing asked on top of it
- ✓ diagnostic-api.test.ts > POST /diagnostic/:id/answers > queues a code task once, in a diagnostic attempt with no hints and no fade stage
- ✓ diagnostic-api.test.ts > POST /diagnostic/:id/complete > waits for the code task, then seeds mastery and fast-tracks the skills it proves
- ✓ diagnostic-api.test.ts > POST /diagnostic/:id/complete > starts a skill only read correctly at rung 2: Predict is skipped, writing still starts at 0
- ✓ diagnostic-api.test.ts > POST /diagnostic/:id/complete > raises a practised skill without lowering it, so the skills built on it unlock
- ✓ diagnostic-api.test.ts > POST /diagnostic/:id/complete > leaves a skill cleared in practice alone, and doesn't ask it
- ✓ diagnostic-api.test.ts > POST /diagnostic/:id/complete > won't let a practice submission stand in for the code task

- ✓ diagnostic-api.test.ts > POST /diagnostic/:id/complete > moves a skill opened before the test (stage 1, never submitted) up to the last stage
- ✓ diagnostic-api.test.ts > POST /diagnostic/:id/complete > stops waiting for a code task the sandbox never ran, and counts it as unobserved
- ✓ diagnostic-api.test.ts > retake > starts a new test only when asked, up to three, never lowering earlier results
- ✓ diagnostic-api.test.ts > retake > lifts a placement on a retake after a failed code task, reusing the open attempt
- ✓ diagnostic-api.test.ts > retake > finishes cleanly for a learner who has already cleared every skill
- ✓ diagnostic-api.test.ts > retake > refreshes only a seed that has faded below the new estimate
- ✓ diagnostic.test.ts > diagnostic schema > allows only one diagnostic in progress per enrollment
- ✓ enrollment-fade.test.ts > enrollment and fade schema > rejects an enrollment in a language the course doesn't offer
- ✓ enrollment-fade.test.ts > enrollment and fade schema > rejects a skill sitting on a fade stage its schedule doesn't have
- ✓ exercise-draft.test.ts > exercise drafts > become templates the bank's harness can run, with the standard closing instruction
- ✓ exercise-draft.test.ts > exercise drafts > turn a predict draft into a rung-1 template whose program is its own reference
- ✓ exercise-draft.test.ts > exercise drafts > refuse a call statement that doesn't take the test's arguments
- ✓ exercise-draft.test.ts > exercise drafts > prompt with the skill, what it may use, the existing titles and the last failure
- ✓ exercise-draft.test.ts > exercise drafts > give Gemini a schema without the dialect tag
- ✓ exercise-templates.test.ts > cpp-basics template bank > covers rungs 1–5 for each of the first five skills, once each
- ✓ exercise-templates.test.ts > cpp-basics template bank > only uses skills that exist in the course
- ✓ exercise-templates.test.ts > cpp-basics template bank > gives rung 1 a program to read and no harness; rungs 2–5 a function to write

- ✓ exercise-templates.test.ts > cpp-basics placement test bank > has exactly one reading item and one code task per course skill
- ✓ exercise-templates.test.ts > cpp-basics placement test bank > asks reading as a program to read, showing only the code
- ✓ exercise-templates.test.ts > cpp-basics placement test bank > asks code tasks as a rung-5 function with hidden tests beyond the shown example
- ✓ exercise-verification.test.ts > exercise verification > verifies when the reference passes and the starter fails
- ✓ exercise-verification.test.ts > exercise verification > rejects an exercise whose reference doesn't pass
- ✓ exercise-verification.test.ts > exercise verification > rejects an exercise whose tests can't tell the starter from a solution
- ✓ exercise-verification.test.ts > exercise verification > runs only the program itself for rung 1 (there is no starter to try)
- ✓ exercise-verification.test.ts > exercise verification > checks content before it is stored: a starter that won't compile, and wrong solutions that slip through
- ✓ exercises-api.test.ts > GET /exercises/:id > shows the exercise and its sample test, never secrets or hidden tests
- ✓ exercises-api.test.ts > GET /exercises/:id > starts one attempt, enrolling the learner and starting the skill at fade stage 1
- ✓ exercises-api.test.ts > GET /exercises/:id > returns a solved attempt as solved instead of starting a new one
- ✓ exercises-api.test.ts > GET /exercises/:id > hides exercises that aren't verified
- ✓ exercises-api.test.ts > GET /exercises/:id > never shows a predict-the-output exercise's answer
- ✓ exercises-api.test.ts > GET /exercises/:id > never opens a placement-test exercise as practice
- ✓ exercises.test.ts > exercises schema > rejects a second exercise with the same content key
- ✓ exercises.test.ts > exercises schema > rejects a rung outside 1 to 5
- ✓ freeze-hints.test.ts > frozen hints > stores levels 1–4 once, and leaves an exercise that already has hints alone

- ✓ freeze-hints.test.ts > frozen hints > asks again when a hint gives the solution away, and skips the exercise if it happens twice
- ✓ generate-exercise.test.ts > exercise generation > stores a draft the sandbox proves, verified and ready to serve
- ✓ generate-exercise.test.ts > exercise generation > tries again after a bad reply, telling the model what was wrong
- ✓ generate-exercise.test.ts > exercise generation > rejects tests a wrong solution passes, tries Pro last, then falls back to the hand-written exercise
- ✓ generate-exercise.test.ts > exercise generation > checks a predict-the-output draft's claimed output by running the program
- ✓ generate-exercise.test.ts > exercise generation > does nothing for a job that has already finished
- ✓ generation-api.test.ts > POST /exercises/generate > queues one job per skill and rung for an author, and returns the same job while it runs
- ✓ generation-api.test.ts > POST /exercises/generate > is for admins and authors only, and refuses while billing is off
- ✓ generation-api.test.ts > POST /exercises/generate > 404s for an unknown skill or job
- ✓ generation-api.test.ts > automatic generation > queues an exercise when the learner's next skill has none, once
- ✓ generation-budget.test.ts > generation budget > stops before calling the model once ₹300 is spent this month, and falls back
- ✓ hints-api.test.ts > POST /exercises/:id/hints > gives the live tutor's hint for the next level only, and moves the level
- ✓ hints-api.test.ts > POST /exercises/:id/hints > blocks a reply that gives the solution away and serves the frozen hint instead
- ✓ hints-api.test.ts > POST /exercises/:id/hints > falls back to a general nudge when the tutor fails and nothing is stored
- ✓ hints-api.test.ts > POST /exercises/:id/hints > refuses a level above the fade stage's ceiling, and logs it
- ✓ hints-api.test.ts > POST /exercises/:id/hints > shows the answer at level 5 without the tutor, and marks the bottom-out
- ✓ hints-api.test.ts > POST /exercises/:id/hints > stops calling the tutor after 20 live hints in a day

- ✓ hints-api.test.ts > POST /exercises/:id/hints > needs an open attempt on a practice exercise
- ✓ hints-budget.test.ts > hint budget > serves frozen hints without calling the tutor once ₹500 is spent this month
- ✓ identity.test.ts > identity schema > treats email as unique and case-insensitive
- ✓ identity.test.ts > identity schema > deletes a user's refresh tokens with the user
- ✓ judge.test.ts > judge > accepts the right output, ignoring line endings and trailing spaces
- ✓ judge.test.ts > judge > gives WA for different output, including an early exit that prints nothing
- ✓ judge.test.ts > judge > gives RE for a crash or a non-zero exit
- ✓ judge.test.ts > judge > gives MLE above the memory limit or on std::bad_alloc
- ✓ judge.test.ts > judge > gives TLE when the time limit killed the test, whatever else happened
- ✓ judge.test.ts > judge > lets the first failing test decide, and calls an empty test list a setup error
- ✓ judge.test.ts > judge > cuts long output to a readable excerpt
- ✓ mastery-api.test.ts > mastery API > lists skills with reading and writing apart, decayed on read
- ✓ mastery-api.test.ts > mastery API > shows one skill in detail, and zeros for a skill never practised
- ✓ mastery-api.test.ts > mastery API > answers 404 for a skill that doesn't exist
- ✓ mastery-formula.test.ts > mastery formula v1 > follows the approved journey: about five good solves pass 0.7
- ✓ mastery-formula.test.ts > mastery formula v1 > gives no credit for an AC after the solution was shown
- ✓ mastery-formula.test.ts > mastery formula v1 > penalises only the first WA or RE of an attempt, more on easy rungs
- ✓ mastery-formula.test.ts > mastery formula v1 > starts TLE/MLE tiny, doubles each time, caps at a WA, and lets only AC reset the streak
- ✓ mastery-formula.test.ts > mastery formula v1 > leaves a counted compile error's score alone

- ✓ mastery-formula.test.ts > mastery formula v1 > gives less credit when much slower than usual, but never below 70%
- ✓ mastery-formula.test.ts > mastery formula v1 > counts rung 1 as reading and rungs 2–5 as writing
- ✓ mastery-formula.test.ts > mastery formula v1 > halves effective mastery after ln 2 / λ days, and never decays without evidence
- ✓ mastery.test.ts > mastery schema > rejects evidence that has no source
- ✓ mastery.test.ts > mastery schema > never counts the same evidence twice
- ✓ placement.test.ts > placement order > puts every prerequisite before the skills built on it, easiest first
- ✓ placement.test.ts > placement walk > ends after one item for a novice who doesn't know, and after the check for one who guesses
- ✓ placement.test.ts > placement walk > asks writing only on the frontier of what was read correctly
- ✓ placement.test.ts > placement walk > places all 711 possible learners exactly, within 20 reading and 7 writing items
- ✓ placement.test.ts > placement walk > rescues a typed slip when the next skill is right, and confirms it when that is wrong too
- ✓ placement.test.ts > placement walk > checks a suspect skill once: its other dependents wait for that check
- ✓ placement.test.ts > placement walk > never lets a writing verdict change which item comes next
- ✓ placement.test.ts > placement walk > infers a skill with no reading item from its dependents, and moves the frontier below a missing writing item
- ✓ placement.test.ts > placement walk > never asks or estimates skills already cleared in practice, but builds on them
- ✓ placement.test.ts > placement estimate > lets one frontier AC vouch for every skill below it, at lower confidence
- ✓ placement.test.ts > placement estimate > treats a pending or failed-to-run task as unobserved, and gives unreached skills nothing
- ✓ progress-api.test.ts > GET /progress > shows every skill's reading and writing apart, as they are today, with a status

- ✓ progress-api.test.ts > GET /progress > 404s for a course that doesn't exist in that language
- ✓ recommendations-api.test.ts > GET /recommendations/next > starts a new learner on the first skill at rung 1, enrolls them, and saves the reasons
- ✓ recommendations-api.test.ts > GET /recommendations/next > moves to rung 2 after the rung-1 exercise is solved
- ✓ recommendations-api.test.ts > GET /recommendations/next > names the next skill without an exercise when it has no content yet
- ✓ recommendations-api.test.ts > GET /recommendations/next > returns null when every skill is cleared and fresh, and 404 for an unknown course
- ✓ recommender.test.ts > recommender rules v1 > starts a new learner on the first skill only
- ✓ recommender.test.ts > recommender rules v1 > unlocks the next skill once a prerequisite is cleared, even if it has decayed a little
- ✓ recommender.test.ts > recommender rules v1 > finishes the skill in progress before a due review, and does due reviews before new skills
- ✓ recommender.test.ts > recommender rules v1 > prefers the learner's goal skill among equals, and reports nothing when the course is done
- ✓ recommender.test.ts > recommender rules v1 > follows the approved journey: rung 1 → 2 → 3 → 4 → 5 in five good solves
- ✓ recommender.test.ts > recommender rules v1 > never goes below the fade stage's easiest rung
- ✓ recommender.test.ts > recommender rules v1 > picks the unfinished attempt first, then unseen at the rung, then harder, then easier
- ✓ research.test.ts > research schema > allows one live consent of each kind, and a new one after revoking
- ✓ scaffolding-api.test.ts > scaffolding fade > moves a skill on when the stage's rung is solved with at most 1 hint, and logs why
- ✓ scaffolding-api.test.ts > scaffolding fade > stays put with too many hints or on an easier rung, and moves one stage at a time
- ✓ scaffolding-api.test.ts > scaffolding fade > never moves past the last stage
- ✓ scaffolding-api.test.ts > scaffolding fade > shows each skill's stage and limits
- ✓ session-clock.test.ts > session clock > leaves a recently active session alone

- ✓ session-clock.test.ts > session clock > pauses after 15 idle minutes, from the moment it went idle
- ✓ session-clock.test.ts > session clock > ends a session paused for over 24 hours, whether paused by hand or by going idle
- ✓ session-clock.test.ts > session clock > counts time between activities as study, but never while paused
- ✓ session-clock.test.ts > session clock > pauses, resumes and ends on request, and does nothing for the same state
- ✓ sessions-api.test.ts > POST /sessions > starts a session on the recommender's first step, and resumes it on the next call
- ✓ sessions-api.test.ts > POST /sessions > ends as completed when the course is done, and 404s for an unknown course
- ✓ sessions-api.test.ts > GET /sessions/active > moves to the next step once the current exercise is solved
- ✓ sessions-api.test.ts > GET /sessions/active > pauses a session idle for 15 minutes without counting the gap, and ends one paused for a day
- ✓ sessions-api.test.ts > PATCH /sessions/:id > pauses, resumes at the same step, and ends for good
- ✓ sessions-api.test.ts > PATCH /sessions/:id > hides other learners' sessions and rejects unknown states
- ✓ sessions.test.ts > sessions schema > allows only one open session per learner
- ✓ sessions.test.ts > sessions schema > allows only one active step per session
- ✓ skill-graph.test.ts > skill graph > finds a cycle
- ✓ skill-graph.test.ts > skill graph > accepts a graph where skills share prerequisites
- ✓ skill-graph.test.ts > skill graph > cpp-basics seed data > has unique slugs and only known prerequisites
- ✓ skill-graph.test.ts > skill graph > cpp-basics seed data > has no cycles
- ✓ solution-filter.test.ts > solution filter > passes a hint in words, and a small fragment of a different idea
- ✓ solution-filter.test.ts > solution filter > blocks the solution, whole or in its key lines, fenced or not
- ✓ solution-filter.test.ts > solution filter > doesn't count code the learner already has

- ✓ solution-filter.test.ts > solution filter > blocks more than a fragment of code, even unrelated code
- ✓ solution-filter.test.ts > solution filter > blocks a predict-the-output hint that gives the output away
- ✓ submissions.test.ts > submissions > saves and queues a submission, answering 202
- ✓ submissions.test.ts > submissions > returns the same submission for a repeated idempotency key, counted once
- ✓ submissions.test.ts > submissions > refuses a key already used on another attempt, so no attempt can borrow another's verdict
- ✓ submissions.test.ts > submissions > hides other learners' attempts and submissions
- ✓ submissions.test.ts > submissions > judges a run, shows the sample test in full and the hidden test as a verdict only
- ✓ submissions.test.ts > submissions > shows compiler output for CE, and doesn't count it before the third
- ✓ submissions.test.ts > submissions > retries a sandbox failure, then gives SE without counting it
- ✓ submissions.test.ts > submissions > turns an AC into writing mastery, saved with its ledger row
- ✓ submissions.test.ts > submissions > penalises only the first wrong answer of an attempt
- ✓ submissions.test.ts > submissions > records no evidence for SE or an early compile error

Real-sandbox smoke test:

```
ok   AC  → AC   5866 ms
ok   WA  → WA   5341 ms
ok   CE  → CE   3565 ms
ok   RE  → RE   5118 ms
ok   TLE → TLE  6965 ms
ok   MLE → MLE  5424 ms
All six verdicts correct.
```

Content verification on the real sandbox (18 October 2026), placement items:

```
Verifying 40 draft exercise(s)…
ok       rung 1 · What does it print? Comments and line breaks
ok       rung 1 · What does it print? Implicit conversions
…        (36 more)
ok       rung 5 · Sort by distance, split near from far
```

```
ok       rung 5 · Own a heap array inside a struct
40 verified, 0 rejected.
```

A-9 Sample Inputs

```
# Register (adult)
curl -X POST http://localhost:3001/api/v1/auth/register -H "Content-
Type: application/json" \
                                                                    -d
'{"email":"adult@test.com","password":"password123","displayName":"Adul
t","dateOfBirth":"2000-05-
01","acceptTerms":true,"acceptPrivacy":true,"aiTraining":true}'

# Submit code for an attempt
curl    -X    POST    http://localhost:3001/api/v1/submissions    -H
"Authorization: Bearer <token>" -H "Content-Type: application/json" \
   -d '{"attemptId":"<attempt id>","answer":"int sumOfDigits(int n) {
int s = 0; for (; n > 0; n /= 10) s += n % 10; return s;
}","idempotencyKey":"try-001"}'

# Placement test: start, then answer the current question (or say "I
don't know")
curl   -X   POST   http://localhost:3001/api/v1/diagnostic/start   -H
"Authorization: Bearer <token>" -H "Content-Type: application/json" -d
'{}'
curl -X POST http://localhost:3001/api/v1/diagnostic/<id>/answers -H
"Authorization: Bearer <token>" -H "Content-Type: application/json" \
  -d '{"itemId":"<item id>","dontKnow":true}'

# Ask for the next hint, sending the current code
curl -X POST http://localhost:3001/api/v1/exercises/<exercise id>/hints
-H "Authorization: Bearer <token>" -H "Content-Type: application/json"
\
  -d '{"code":"int sumOfDigits(int n) { return n % 10; }"}'
```

A-10 Sample Outputs

Real responses captured during testing (IDs shortened):

```
// Register (adult)
{"success":true,"data":{"user":
{"email":"adult@test.com","displayName":"Adult","role":"LEARNER",
 "dateOfBirth":"2000-05-01","status":"active"}}}

// Register (12 years old), then log in
{"success":true,"data":{"user":
{"email":"kid@test.com","status":"pending_parent"}}}
{"success":false,"error":{"code":"PARENT_APPROVAL_PENDING","message":"A
parent or guardian needs to approve this account first"}}

// Submission result: wrong answer (return n % 10)
{"verdict":"WA","testsPassed":1,"testsTotal":4,"tests":[
```

```
{"seq":1,"name":"Example","visibility":"sample","verdict":"WA","input":
"123","expectedOutput":"6","output":"3\n"},

{"seq":2,"name":"Zero","visibility":"hidden","verdict":"AC","timeMs":0}
,
                                                    {"seq":3,"name":"All
nines","visibility":"hidden","verdict":"WA","timeMs":0},
                                                  {"seq":4,"name":"Largest
input","visibility":"hidden","verdict":"WA","timeMs":0}]}

// Submission result: correct answer
{"verdict":"AC","testsPassed":4,"testsTotal":4}
```

```
End-to-end learner loop (16 Oct 2026)
GET exercise:   Sum of digits | rung 3 | skill Functions | sampleTests
1 | attempt active, ceiling 5
verdict:        AC 4/4
GET mastery:    Functions | writing 0.240 effective 0.240 | attempts 1
reopen attempt: solved
```

```
Placement test on real E2B (18 Oct 2026): a learner who knows the first
3 skills
start: attempt 1, first question: reading program-structure    (the
answer is not in the response)
  reading program-structure      → answers
  reading variables-and-types    → answers
  reading operators              → answers
  reading conditionals           → I don't know
  reading bitwise-operators      → I don't know
   writing operators                    → answers (code, run in the
background)
answered 6 in 621 ms; complete → 202 grading, retry in 5000 ms →
completed after 5676 ms
  program-structure      reading 0.85 writing 0.85 confidence 0.5
  variables-and-types    reading 0.85 writing 0.85 confidence 0.5
  operators              reading 0.85 writing 0.85 confidence 1
next step: conditionals, rung 1
fade: program-structure = stage 4, variables-and-types = stage 4,
operators = stage 4
```

```
Session and progress (19 Oct 2026)
start:  active | step 1 Your first program, rung 1
pause:  paused | resume: active, same step
progress summary: total 20, locked 19, ready 1, in progress 0, cleared
0, review due 0
end:    user | active session after end: none
```

```
Hint ladder with live AI switched off (19 Oct 2026), a rung-1 exercise
200 level 1/5 generic: Re-read the task and list what goes in and what
must come out…
200 level 2/5 generic: Run the example by hand, line by line…
```

```
200 level 3/5 generic: Change one thing at a time and run again…
200 level 4/5 generic: Solve a tiny version of the task first…
200 level 5/5 answer:  It prints: Hello World / 3. Trace it again line
by line…
409 HINT_LIMIT
```

```
Admin overview, last 30 days (19 Oct 2026, development database)
learners: total 9, joined 9, ran code 4, waiting for a parent 1, AI-
training opt-ins 1
runs:     5 submissions (AC 4, WA 1), median run 50 ms, sandbox error
rate 0
content:  14 of 20 skills have no practice exercises yet; Your first
program has one per rung
```

```
Exercise generation with live AI switched off (19 Oct 2026), as an
admin
POST /exercises/generate { skill: "vectors", rung: 2 } → 503
GENERATION_UNAVAILABLE
```

Screen snapshots of the web interface will be added here in the final report.

A-11 Coding (selected)

The verdict judge (`apps/api/src/modules/m7_execution/services/judge.ts`)

```
import type { verdict } from
"../../../infrastructure/database/schema/index.js";
import type { Limits, TestRun } from
"../../../infrastructure/sandbox/sandbox-provider.js";

// Turns raw sandbox readings into verdicts (ADR-0003, "Verdict
mapping"). Pure: no sandbox, no database.
export type Verdict = (typeof verdict.enumValues)[number];

const EXCERPT_CHARS = 4000;

// Same output once line endings, trailing spaces and blank lines at
the end are ignored.
const normalise = (text: string) =>
  text
    .replace(/\r\n/g, "\n")
    .split("\n")
    .map((line) => line.trimEnd())
    .join("\n")
    .trimEnd();

export const sameOutput = (actual: string, expected: string) =>
normalise(actual) === normalise(expected);

export function judgeTest(run: TestRun, expected: string, limits:
Limits): Verdict {
  if (run.timedOut) return "TLE";
```

```
      if (run.peakMemoryKb > limits.memoryLimitMb * 1024 ||
run.stderr.includes("std::bad_alloc")) return "MLE";
  if (run.exitCode !== 0) return "RE";
  return sameOutput(run.stdout, expected) ? "AC" : "WA";
}

// The first failing test decides. No tests means a broken setup, never
the learner's fault.
export const overallVerdict = (testVerdicts: Verdict[]): Verdict =>
  testVerdicts.length ? (testVerdicts.find((v) => v !== "AC") ?? "AC")
: "SE";

// Compiler output and test output are stored and shown cut to a
readable size.
export const excerpt = (text: string) =>
   text.length > EXCERPT_CHARS ? `${text.slice(0, EXCERPT_CHARS)}\n…
(truncated)` : text;
```

The mastery formula (`apps/api/src/modules/m4_mastery/services/mastery-formula.ts`)

```
import type { Verdict } from "../../m7_execution/services/judge.js";

// The M4 formula, v1 (ADR-0004, approved 15 Oct). Pure: no database.
Every evidence row stores
//  FORMULA_VERSION, so old numbers stay explainable when these
constants are tuned.
export const FORMULA_VERSION = "m4-v1";
const GAIN = 0.4; // AC moves the score 40% of the way to 1, scaled by
weight
const PENALTY = 0.15; // WA/RE take 15% of the score, scaled by
failWeight
const HINT_COST = 0.15; // each hint used cuts the AC credit by 15%
const BOTTOM_OUT_LEVEL = 5; // hint 5 shows the solution: no credit
const LIMIT_START = PENALTY / 20; // TLE/MLE start at 1/20 of a WA,
doubling, capped at a WA
const USUAL_MINUTES = [5, 8, 12, 15, 20]; // per rung; placeholders
until real timings exist
const SLOWEST_TIME_FACTOR = 0.7;

export type Evidence = {
  verdict: Verdict;
  rung: number;
  hintsUsed: number;
   activeMs: number; // 0 until sessions track active time; then time
counts
  isRepeatFailure: boolean; // an earlier WA/RE in the same attempt was
already penalised
};

export type Update = { score: number; weight: number; limitStreak:
number };

// Rung 1 is "predict the output", so it's evidence about reading code;
```

```
rungs 2–5 about writing it.
export const dimensionOf = (rung: number) => (rung === 1 ? "reading" :
"writing");

export function applyEvidence(before: number, limitStreak: number, e:
Evidence): Update {
  const unchanged = { score: before, weight: 0, limitStreak };
  const failWeight = (6 - e.rung) / 5; // failing an easy exercise says
more than failing a hard one

  switch (e.verdict) {
    case "AC": {
      const hints = e.hintsUsed >= BOTTOM_OUT_LEVEL ? 0 : 1 - HINT_COST
* e.hintsUsed;
      const weight = (e.rung / 5) * hints * timeFactor(e);
       return { score: before + GAIN * weight * (1 - before), weight,
limitStreak: 0 };
    }
    case "WA":
    case "RE":
       return e.isRepeatFailure ? unchanged : { score: before * (1 -
PENALTY * failWeight), weight: failWeight, limitStreak };
    case "TLE":
    case "MLE": {
      const penalty = Math.min(LIMIT_START * 2 ** limitStreak, PENALTY)
* failWeight;
        return { score: before * (1 - penalty), weight: failWeight,
limitStreak: limitStreak + 1 };
    }
    default:
      return unchanged; // a counted CE: it shows in attempt totals but
doesn't move the score
  }
}

// Slower than the rung's usual time lowers credit, never below 0.7. No
time tracked = full credit.
function timeFactor({ rung, activeMs }: Evidence) {
  if (!activeMs) return 1;
  const usualMs = USUAL_MINUTES[rung - 1] * 60_000;
     return   Math.min(1,   Math.max(SLOWEST_TIME_FACTOR,   usualMs   /
activeMs));
}

const DAY_MS = 86_400_000;

// FR-M4-03: what the learner can still do today. Stored scores never
decay; this is computed on read.
export const effectiveScore = (score: number, decayLambda: number,
evidenceAt: Date | null, now = new Date()) =>
   evidenceAt ? score * Math.exp((-decayLambda * (now.getTime() -
evidenceAt.getTime())) / DAY_MS) : score;
```

The hint leak filter (`apps/api/src/modules/m8_socratic/services/solution-filter.ts`)

```
// FR-M8-04: every generated hint is checked before a learner sees it. Pure, so it is unit-tested.
// Code is compared as overlapping runs of 5 tokens ("fingerprints"). A hint is blocked when it
// carries too many of the fingerprints only the solution has (not the starter, not the learner's
// own code), when it shows more than a fragment of code, or when it gives away a rung-1 output.
const RUN = 5;
const MAX_SHARED = 0.25; // share of the solution's own fingerprints a hint may contain
const MAX_CODE_LINES = 6; // a hint may show a fragment, never a program
const MIN_OUTPUT_CHARS = 4; // shorter outputs ("3") appear in prose by chance

export type FilterContext = { reference: string; starter: string; learnerCode?: string; output?: string };
export type FilterResult = { passed: boolean; shared: number; longestCode: number; givesOutput: boolean };

const TOKEN = /[A-Za-z_]\w*|\d+|"(?:\\.|[^"\\])*"|'(?:\\.|[^'\\])*'|\S/g;
const tokens = (code: string) => code.replace(/\/\/.*$/gm, "").replace(/\/\*[\s\S]*?\*\//g, "").match(TOKEN) ?? [];
const fingerprints = (text: string) => {
  const t = tokens(text);
  return new Set(Array.from({ length: Math.max(0, t.length - RUN + 1) }, (_, i) => t.slice(i, i + RUN).join(" ")));
};
const codeBlocks = (reply: string) => [...reply.matchAll(/```[^\n]*\n([\s\S]*?)```/g)].map((m) => m[1]);
const squash = (text: string) => text.replace(/\s+/g, " ").trim();

export function checkHint(reply: string, { reference, starter, learnerCode = "", output }: FilterContext): FilterResult {
  const known = new Set([...fingerprints(starter), ...fingerprints(learnerCode)]);
  const own = [...fingerprints(reference)].filter((f) => !known.has(f));
  const inReply = fingerprints(reply);
  const shared = own.length ? own.filter((f) => inReply.has(f)).length / own.length : 0;
  const longestCode = Math.max(0, ...codeBlocks(reply).map((c) => c.trim().split("\n").length));
  const answer = output ? squash(output) : "";
  const givesOutput = answer.length >= MIN_OUTPUT_CHARS && squash(reply).includes(answer);
  return { passed: shared < MAX_SHARED && longestCode <= MAX_CODE_LINES && !givesOutput, shared, longestCode, givesOutput };
}
```